R3.04 – Qualité de développement

14 – Design patterns

Sébastien Thon

BUT Informatique 2ème année
IUT d'Aix-Marseille Université, site d’Arles
Département Informatique

Objectifs du Cours

- Comprendre les concepts de base des design patterns
- Explorer les différents types de design patterns
- Apprendre à implémenter des design patterns en Java
- Améliorer la maintenabilité et la réutilisabilité du code

Qu'est-ce qu'un Design Pattern ?

- Un design pattern est une solution réutilisable à un problème commun rencontré dans le développement logiciel.

- Il s'agit d'une description ou d'un modèle utilisé pour résoudre un problème qui peut se produire dans de multiples situations différentes.

- Les design patterns ne sont pas des algorithmes ou des codes spécifiques, mais plutôt des concepts généraux que vous pouvez appliquer pour résoudre des problèmes de conception.

Importance des Design Patterns

- Les design patterns offrent un langage commun pour les développeurs, facilitant la communication et la compréhension des solutions de conception.

- Ils permettent d'améliorer la flexibilité et la réutilisabilité du code, réduisant ainsi les coûts de maintenance.

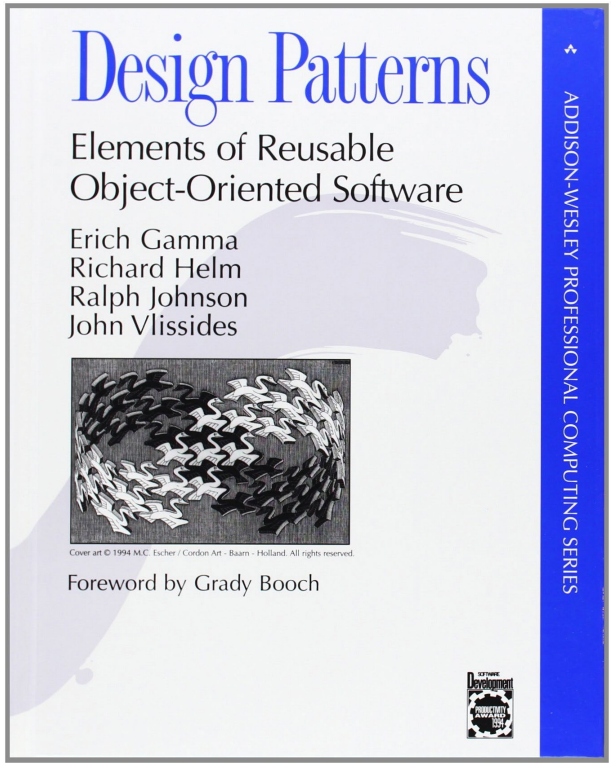
- En utilisant des design patterns éprouvés, les développeurs peuvent éviter de réinventer la roue et se concentrer sur les aspects spécifiques de leur projet.

Historique des Design Patterns

- L'idée des design patterns a été popularisée par le livre "Design Patterns: Elements of Reusable Object-Oriented Software" publié en 1994 par Erich Gamma, Richard Helm, Ralph Johnson et John Vlissides (connus sous le nom de "Gang of Four" ou GoF).

- Ce livre a catalogué 23 design patterns couramment utilisés et a posé les bases de la conception de logiciels orientés objet.

- Depuis, de nombreux autres patterns ont été identifiés et documentés par la communauté des développeurs, et les design patterns sont devenus une partie essentielle de l'ingénierie logicielle.



Classification des Design Patterns

1. Patterns de **Création**
   - permettent la création d'objets de manière contrôlée.
   - Abstraient le processus d'instanciation et masquent la logique de création.

2. Patterns **Structurels**
   - se concentrent sur la façon dont les classes et les objets sont composés pour former des structures plus complexes tout en gardant ces structures flexibles et efficaces.

3. Patterns **Comportementaux**
   - se concentrent sur les algorithmes et les responsabilités entre les objets.
   - décrivent comment les objets interagissent les uns avec les autres, comment ils communiquent et comment les responsabilités sont distribuées entre eux.

Ne pas confondre

- **Architecture pattern :** patrons architecturaux qui décrivent la structure globale d'une application, son organisation.
  - MVC (Model-View-Controller)
  - MVVM (Model-View-ViewModel)
  - MVP (Model-View-Presenter)
  - …

- **Design pattern** : ils sont de plus bas niveau, au niveau des algorithmes.

Rappel sur les interfaces en Java

- Les design patterns en Java utilisent énormément les **interfaces**.
- En Java, une interface sert à définir un contrat que des classes doivent respecter, sans donner directement d'implémentation.
- Une interface décrit ce qu'une classe doit faire, mais pas comment elle le fait.
- Les classes qui implémentent une interface doivent fournir le code des méthodes définies dans cette interface.

Une interface se présente comme ci-dessous : le mot-clé **`interface`** suivi de son nom, puis, entre accolades, une liste de **constantes** (mais pas de variable), de **prototypes** de méthodes, de **méthodes statiques**, ou de méthodes par défaut (default) qui possèdent une implémentation :

```
interface Paiement {
   // Une constante (sera implicitement : public static final)
   double FRAIS_TRANSACTION = 0.5;

   // Méthode abstraite (contrat)
   void payer(double montant);

   // Méthode statique (accessible sans objet)
   static void afficherConditions() {
      System.out.println("Tous les paiements incluent des frais fixes de "
                  + FRAIS_TRANSACTION + "€.");
   }

   // Méthode par défaut (possède déjà une implémentation)
   default void annuler() {
      System.out.println("Le paiement a été annulé.");
   }
}
```

Ensuite, différentes classes **implémentent** cette interface selon leur mode de fonctionnement :

CarteBancaire.java

```
class CarteBancaire implements Paiement {
    public void payer(double montant) {
        System.out.println("Paiement de " + (montant + FRAIS_TRANSACTION)
                + "€ par carte bancaire.");
    }
}
```

PayPal.java

```
class PayPal implements Paiement {
    public void payer(double montant) {
        System.out.println("Paiement de " + (montant + FRAIS_TRANSACTION)
                + "€ via PayPal.");
    }
}
```

On peut ensuite utiliser n'importe quel moyen de paiement de manière uniforme, sans se soucier de la classe exacte :

Magasin.java

```
public class Magasin {
    public static void main(String[] args) {
        // Appel de la méthode statique de l'interface
        Paiement.afficherConditions();

        Paiement paiement;                  // la variable est de type Paiement
        paiement = new CarteBancaire();  // …mais l'objet réel est une CarteBancaire
        paiement.payer(50.0);               // Paiement de 50.5€ par carte bancaire
        paiement.annuler();                 // Appel de la méthode par défaut

        paiement = new PayPal();            // …puis l'objet réel est un PayPal
        paiement.payer(75.0);               // Paiement de 75.5€ via PayPal
    }
}
```

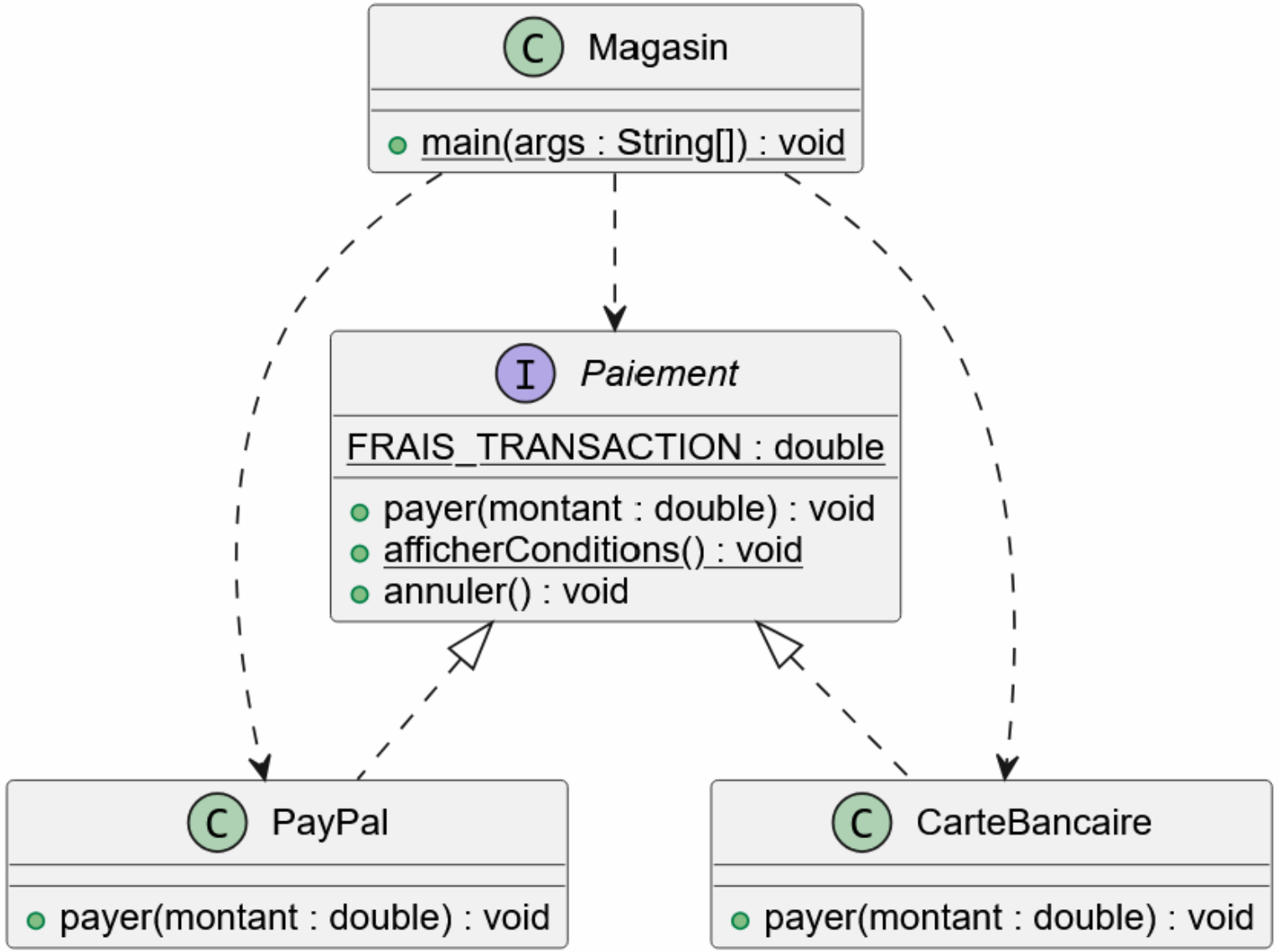
Si on veut ajouter un nouveau mode de paiement (ex : Google Pay), il suffit de créer une nouvelle classe qui implémente **Paiement**.

```
Paiement paiement = new CarteBancaire();        // Upcasting implicite
paiement.payer(50.0);                           // Appelle la méthode payer() de CarteBancaire
```

- On ne peut **pas** instancier l'interface : ~~Paiement paiement = new Paiement()~~
- On peut référencer un objet d'une classe concrète par le **type interface** qu'elle implémente.
- Via la variable **paiement**, on n'accède **qu'aux méthodes déclarées dans l'interface**.
- Si CarteBancaire a une méthode spécifique non présente dans Paiement, il faudra caster :

```
((CarteBancaire) paiement).activerSansContact();
```

→ ce principe permet d'écrire du code générique sur Paiement tout en branchant différentes implémentations (CarteBancaire, Paypal, etc.).


Magasin
main(args : String[]) : void
Paiement
FRAIS_TRANSACTION : double
payer(montant : double) : void
afficherConditions() : void
annuler() : void
PayPal
payer(montant : double) : void
CarteBancaire
payer(montant : double) : void

→ Cet exemple correspond au design pattern **Strategy** qui est un **pattern comportemental**.

Remplace des if/else d’algorithmes par des classes interchangeables.

Utile quand il y a plusieurs variantes d’un comportement (ici, de paiement).

Dans ce qui suit, je présente les 23 design patterns définis par le GoF :

- 5 **patterns créationnels** : Abstract Factory, Builder, Factory Method, Prototype, Singleton.

- 7 **patterns structurels** : Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy.

- 11 **patterns comportementaux** : Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor.

Les patterns de création

Objectifs :

- Les patterns de création sont des mécanismes de conception qui permettent la création d'objets de manière contrôlée.

- Ils abstraient le processus d'instanciation et masquent la logique de création.

Singleton

Définition :

- Le pattern **Singleton** est un **pattern de création** qui garantit qu'une classe n'a qu'une seule instance et fournit un point d'accès global à cette instance.

- Il est utilisé lorsque vous devez contrôler l'accès à certaines ressources partagées, comme une base de données ou une connexion réseau.

Singleton

Cas d'utilisation :

- Gestion des connexions aux bases de données : Assurer qu'il n'y a qu'une seule connexion active à la fois.
- Gestion des configurations : Fournir un accès centralisé aux paramètres de configuration d'une application.
- Logger des informations sur l’application.

Exemple

```
public class Singleton {
	private static Singleton instance;
	private int compteur = 0;
	// Constructeur privé pour empêcher
	 // l'instanciation externe
	private Singleton() {	}

	public static Singleton getInstance() {
		if (instance == null) {
			instance = new Singleton();
		}
		return instance;
	}

	public void incrementer() {
		compteur++;
	}

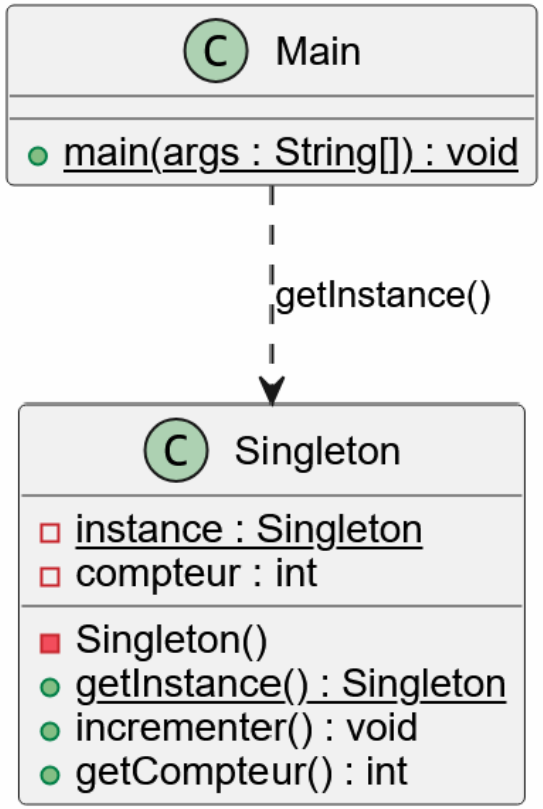
	public int getCompteur() {
		return compteur;
	}
}
```

```
// Exemple d'utilisation
public class Main {
   public static void main(String[] args) {
      Singleton s1 = Singleton.getInstance();
      Singleton s2 = Singleton.getInstance();

      s1.incrementer();
      s2.incrementer();

      System.out.println("Compteur S1 = " + s1.getCompteur());
      // Affiche : Compteur S1 = 2
      System.out.println("Compteur S2 = " + s2.getCompteur());
      // Affiche : Compteur S2 = 2

      System.out.println(s1 == s2);
      // Affiche : true
   }
}
```


Main
main(args : String[]) : void
getInstance()
Singleton
instance : Singleton
compteur : int
Singleton()
getInstance() : Singleton
incrementer() : void
getCompteur() : int

Singleton

Avantages:

- Contrôle strict sur l'instanciation : Assure qu'il n'y a qu'une seule instance de la classe.
- Accès global : Fournit un point d'accès centralisé pour l'instance.
- Réduction de l'utilisation des ressources : Empêche la création d'instances multiples, économisant ainsi les ressources système.

Singleton

Inconvénients:

- Difficulté de test : Le pattern Singleton peut rendre les tests unitaires plus complexes en raison de la difficulté à isoler l'instance unique.

- Potentiel de goulot d'étranglement : Si l'instance unique devient un point de contention dans l'application, cela peut affecter les performances.

- Couplage fort : L'utilisation excessive de Singletons peut entraîner un couplage fort entre les classes, rendant le code moins flexible.

- …car un Singleton est souvent appelé directement depuis beaucoup de classes, par exemple :

```
class Commande {
    public void traiter() {
        Logger.getInstance().log("Commande traitée");
    }
}
```

→ **Commande** est fortement couplée à **Logger**. Si on veut plus tard remplacer **Logger** par un autre système, ou utiliser un faux logger pour les tests, on doit modifier **Commande**.

→ **Le Singleton est pratique, mais utilisé partout = dépendances cachées et code moins facile à faire évoluer.**

Une solution : une dépendance passée de l'extérieur :

```
class Commande {
    private Logger logger;

    public Commande(Logger logger) {
        this.logger = logger;
    }

    public void traiter() {
        logger.log("Commande traitée");
    }
}
```

→la classe **Commande** est plus flexible, car on peut lui fournir un autre objet.

Principe d'**injection de dépendances** (*Dependency Injection*, DI). Au lieu qu'une classe crée ou récupère elle-même sa dépendance, on la lui fournit depuis l'extérieur (ici via son constructeur).

Factory Method

Définition :

- Le pattern **Factory Method** est un **pattern de création** qui fournit une interface pour créer des objets dans une classe mère, mais permet aux sous-classes de modifier le type d'objets qui seront créés.

- Il permet de créer des objets sans spécifier la classe exacte de l'objet à créer.

Factory Method

Cas d'utilisation :

- Lorsqu'une classe ne peut pas anticiper le type d'objets qu'elle doit créer.
- Lorsque les responsabilités de création d'objets doivent être déléguées à des sous-classes.
- Lorsqu'il est nécessaire de fournir des implémentations différentes d'une même interface ou classe abstraite.

Exemple

```
// Interface Produit
public interface Produit {
    void operation();
}

// Implémentations concrètes de Produit
public class ProduitA implements Produit {
    public void operation() {
        System.out.println("Produit A");
    }
}

public class ProduitB implements Produit {
    public void operation() {
        System.out.println("Produit B");
    }
}
```

```
// Classe créatrice abstraite
public abstract class Createur {
    public abstract Produit factoryMethod();

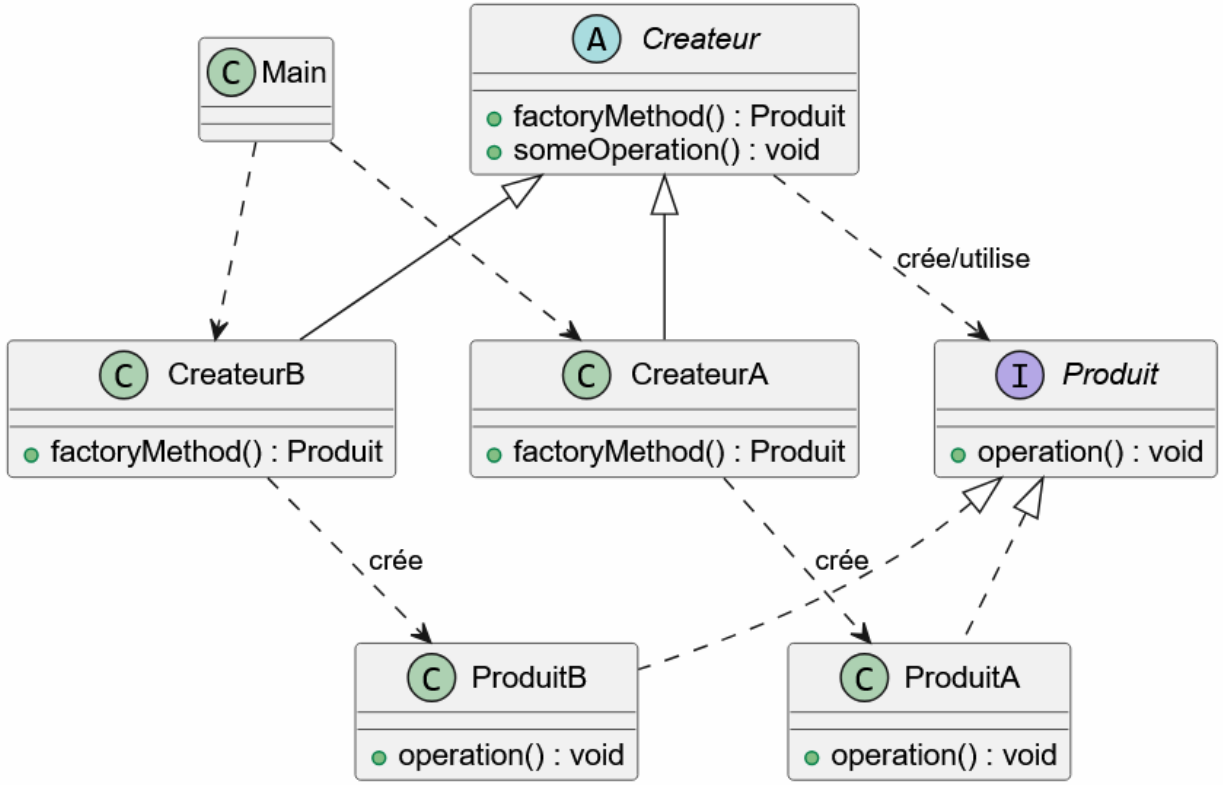
    public void someOperation() {
        Produit produit = factoryMethod();
        produit.operation();
    }
}

// Créateurs concrets
public class CreateurA extends Createur {
    public Produit factoryMethod() {
        return new ProduitA();
    }
}

public class CreateurB extends Createur {
    public Produit factoryMethod() {
        return new ProduitB();
    }
}
```

```
// Utilisation
public class Main {
    public static void main(String[] args) {
        Createur createurA = new CreateurA();
        createurA.someOperation();

        Createur createurB = new CreateurB();
        createurB.someOperation();
    }
}
```



On aurait pu utiliser ce design pattern pour le TP3 de gestion de médiathèque.

On aurait pu utiliser ce design pattern pour le TP3 de gestion de médiathèque :

```
abstract class Ouvrage {
    private final int id;
    protected boolean emprunte;
    protected Ouvrage(int id) { this.id = id; }
    public abstract void afficher();
}

class Livre extends Ouvrage {
    private final String titre, auteur;
    public Livre(int id, String titre, String auteur) { super(id); this.titre = titre; this.auteur = auteur; }
    @Override public void afficher() { System.out.println("[Livre] " + titre + " - " + auteur); }
}

class DVD extends Ouvrage {
    private final String titre; private final int duree;
    public DVD(int id, String titre, int duree) { super(id); this.titre = titre; this.duree = duree; }
    @Override public void afficher() { System.out.println("[DVD] " + titre + " - " + duree + " min"); }
}
```

```
abstract class OuvrageCreator {
   // "factory method" à surcharger
   protected abstract Ouvrage factoryMethod(java.util.Scanner sc);

   public Ouvrage creerDepuisConsole(java.util.Scanner sc) {
      return factoryMethod(sc);
   }
}

class LivreCreator extends OuvrageCreator {
   @Override protected Ouvrage factoryMethod(java.util.Scanner sc) {
      System.out.print("Id ? ");   int id = Integer.parseInt(sc.nextLine());
      System.out.print("Titre ? "); String titre = sc.nextLine();
      System.out.print("Auteur ? "); String auteur = sc.nextLine();
      return new Livre(id, titre, auteur);
   }
}

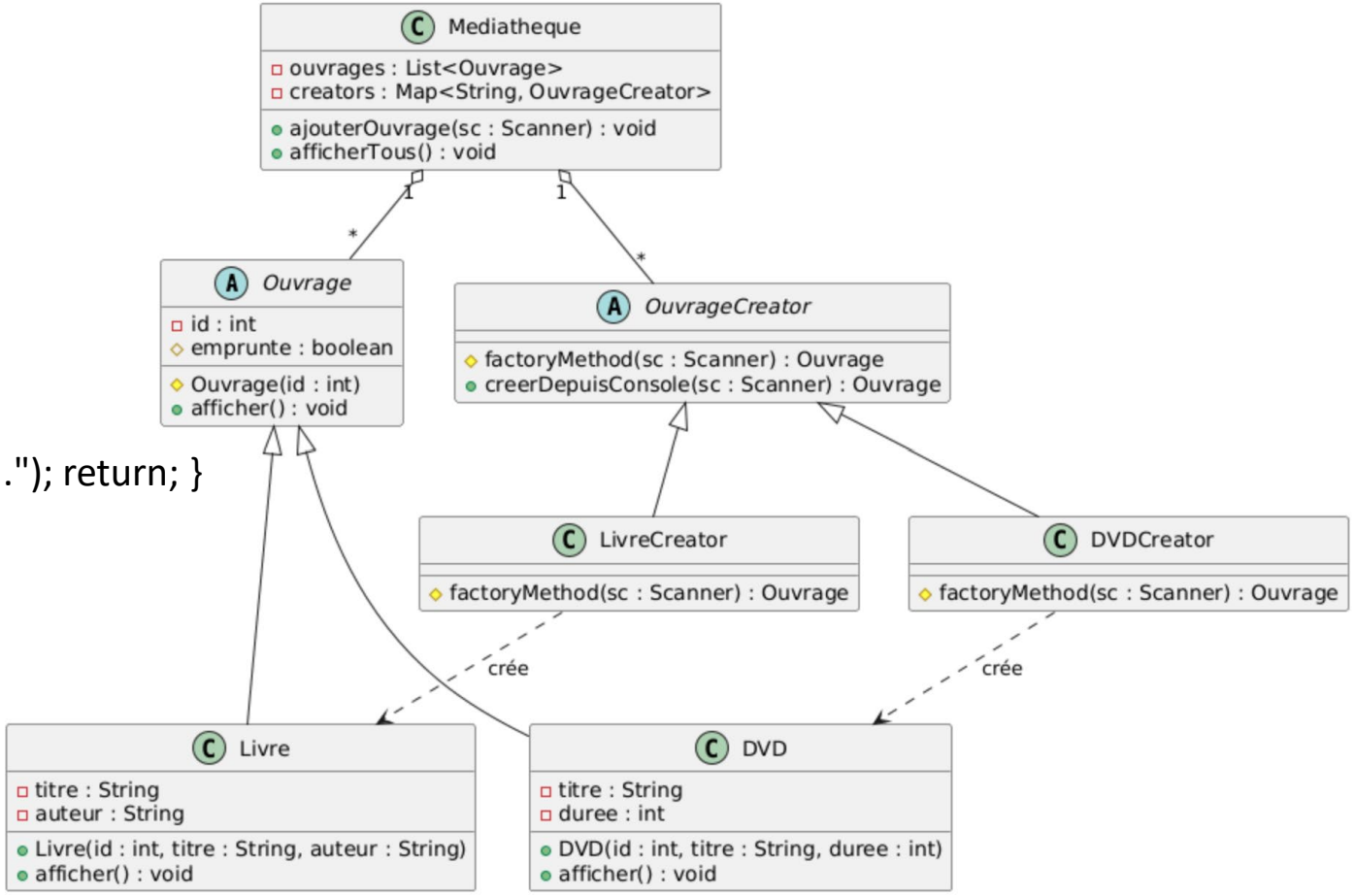
class DVDCreator extends OuvrageCreator {
   @Override protected Ouvrage factoryMethod(java.util.Scanner sc) {
      System.out.print("Id ? ");   int id = Integer.parseInt(sc.nextLine());
      System.out.print("Titre ? "); String titre = sc.nextLine();
      System.out.print("Durée (min) ? "); int duree = Integer.parseInt(sc.nextLine());
      return new DVD(id, titre, duree);
   }
}
```

```
class Mediatheque {
   private final java.util.List<Ouvrage> ouvrages = new java.util.ArrayList<>();

  private final java.util.Map<String,OuvrageCreator> creators = java.util.Map.of(
      "livre", new LivreCreator(),
      "dvd",   new DVDCreator()
   );

   public void ajouterOuvrage(java.util.Scanner sc) {
      System.out.print("Type (livre/dvd) ? ");
      String type = sc.nextLine().toLowerCase();
      OuvrageCreator creator = creators.get(type);
      if (creator == null) { System.out.println("Type inconnu."); return; }
      ouvrages.add(creator.creerDepuisConsole(sc));
   }

   public void afficherTous() {
      ouvrages.forEach(Ouvrage::afficher);
   }
}
```



Factory Method

Avantages :

- Isolation des détails de création : la logique de création des objets est isolée dans les sous-classes, rendant le code plus modulaire et facile à maintenir.

- Extension facile : ajouter de nouveaux types de produits est simple et ne nécessite pas de modification de la logique existante.

- Réduction du couplage : Les classes utilisent l'interface de produit au lieu de classes concrètes, réduisant ainsi le couplage.

Factory Method

Inconvénients :

- Complexité accrue : l'utilisation du pattern Factory Method peut ajouter une complexité supplémentaire en introduisant de nouvelles classes et méthodes.

- Surcharge de code : pour des cas simples, l'utilisation du Factory Method peut être perçue comme une surcharge inutile de code.

Abstract Factory

Définition :

- Le pattern **Abstract Factory** est un **pattern de création** qui fournit une interface pour créer des familles d'objets liés ou dépendants sans spécifier leurs classes concrètes.

- Il permet à une classe de déléguer l'instanciation d'objets à une autre classe en définissant une interface pour créer des objets de différentes familles de produits.

Abstract Factory

Cas d’utilisation :

- Lorsque le système doit être indépendant de la façon dont ses produits sont créés, composés et représentés.
- Lorsqu'un système doit être configuré avec une ou plusieurs familles de produits.
- Lorsque vous souhaitez fournir une bibliothèque de produits et révéler uniquement leurs interfaces, sans exposer les implémentations concrètes.

Exemple

```
// Interfaces des produits
public interface AbstractProduitA {
    void operationA();
}

// Implémentations concrètes des produits
public class ProduitA1 implements AbstractProduitA {
    public void operationA() {
        System.out.println("ProduitA1 opération");
    }
}

public class ProduitA2 implements AbstractProduitA {
    public void operationA() {
        System.out.println("ProduitA2 opération");
    }
}
```

```
// Interfaces des produits
public interface AbstractProduitB {
    void operationB();
}

public class ProduitB1 implements AbstractProduitB {
    public void operationB() {
        System.out.println("ProduitB1 opération");
    }
}

public class ProduitB2 implements AbstractProduitB {
    public void operationB() {
        System.out.println("ProduitB2 opération");
    }
}
```

```
// Interface de la fabrique abstraite
public interface AbstractFactory {
    AbstractProduitA createProduitA();
    AbstractProduitB createProduitB();
}

// Implémentations concrètes de la fabrique
public class ConcreteFactory1 implements AbstractFactory {
    public AbstractProduitA createProduitA() {
        return new ProduitA1();
    }
    public AbstractProduitB createProduitB() {
        return new ProduitB1();
    }
}

public class ConcreteFactory2 implements AbstractFactory {
    public AbstractProduitA createProduitA() {
        return new ProduitA2();
    }
    public AbstractProduitB createProduitB() {
        return new ProduitB2();
    }
}
```

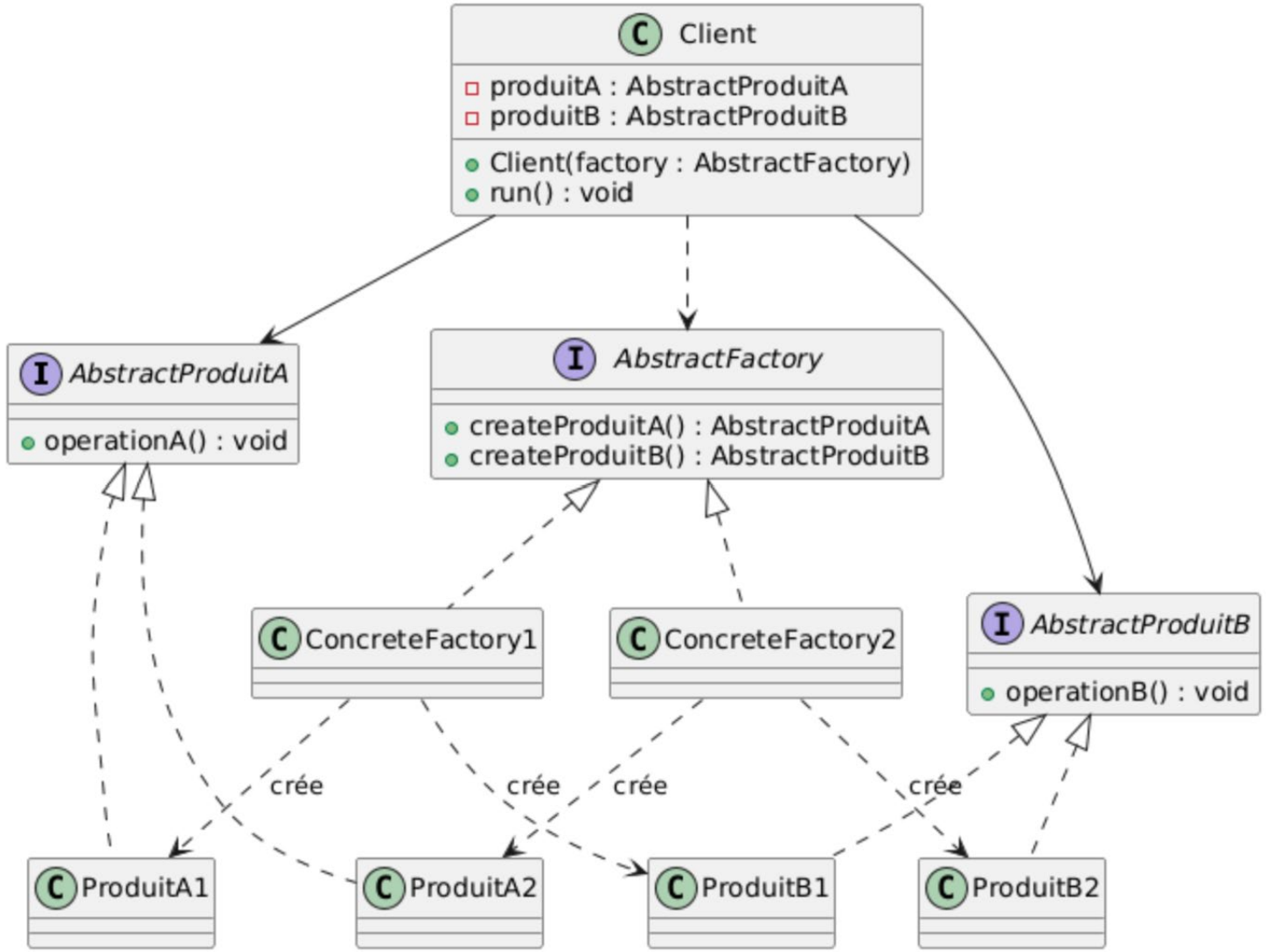
```
// Utilisation
public class Client {
    private AbstractProduitA produitA;
    private AbstractProduitB produitB;

    public Client(AbstractFactory factory) {
        produitA = factory.createProduitA();
        produitB = factory.createProduitB();
    }

    public void run() {
        produitA.operationA();
        produitB.operationB();
    }

    public static void main(String[] args) {
        AbstractFactory factory1 = new ConcreteFactory1();
        Client client1 = new Client(factory1);
        client1.run();

        AbstractFactory factory2 = new ConcreteFactory2();
        Client client2 = new Client(factory2);
        client2.run();
    }
}
```


Client
produitA : AbstractProduitA
produitB : AbstractProduitB
Client(factory : AbstractFactory)
run() : void
AbstractProduitA
operationA() : void
AbstractFactory
createProduitA() : AbstractProduitA
createProduitB() : AbstractProduitB
AbstractProduitB
operationB() : void
ConcreteFactory1
ConcreteFactory2
crée
crée
crée
crée
ProduitA1
ProduitA2
ProduitB1
ProduitB2

Abstract Factory

Avantages :

- Isolation des clients des classes concrètes : Les clients manipulent les interfaces des produits et des fabriques abstraites, ce qui permet de changer facilement les classes concrètes sans affecter le client.

- Cohérence entre les objets : Assure que les produits créés par une fabrique sont compatibles les uns avec les autres.

- Flexibilité et extensibilité : Facile d'ajouter de nouvelles familles de produits sans modifier le code existant.

Abstract Factory

Inconvénients :

- Complexité accrue : Introduit un nombre important de classes et d'interfaces supplémentaires.

- Configuration complexe : Peut nécessiter une configuration complexe pour assembler les fabriques et leurs produits associés.

Builder

Définition :

- Le pattern **Builder** est un **pattern de création** qui permet de construire des objets complexes en séparant le processus de construction de leur représentation. Le même processus de construction peut créer différentes représentations.

- Il est particulièrement utile lorsque la création d'un objet implique plusieurs étapes ou lorsqu'un objet peut avoir différentes configurations.

Exemple

```
// Produit complexe
public class Produit {
   private String partie1;
   private String partie2;
   private String partie3;

   public void setPartie1(String partie1) {
      this.partie1 = partie1;
   }

   public void setPartie2(String partie2) {
      this.partie2 = partie2;
   }

   public void setPartie3(String partie3) {
      this.partie3 = partie3;
   }

   @Override
   public String toString() {
      return "Produit [partie1=" + partie1 + ", partie2=" + partie2 + ", partie3=" + partie3 + "]";
   }
}
```

```
// Interface du Builder
public interface Builder {
   void construirePartie1();
   void construirePartie2();
   void construirePartie3();
   Produit getProduit();
}
```

```java
// Implémentation concrète du Builder
public class ConcreteBuilder implements Builder {
    private Produit produit;

    public ConcreteBuilder() {
        this.produit = new Produit();
    }

    public void construirePartie1() {
        produit.setPartie1("Partie 1");
    }

    public void construirePartie2() {
        produit.setPartie2("Partie 2");
    }

    public void construirePartie3() {
        produit.setPartie3("Partie 3");
    }

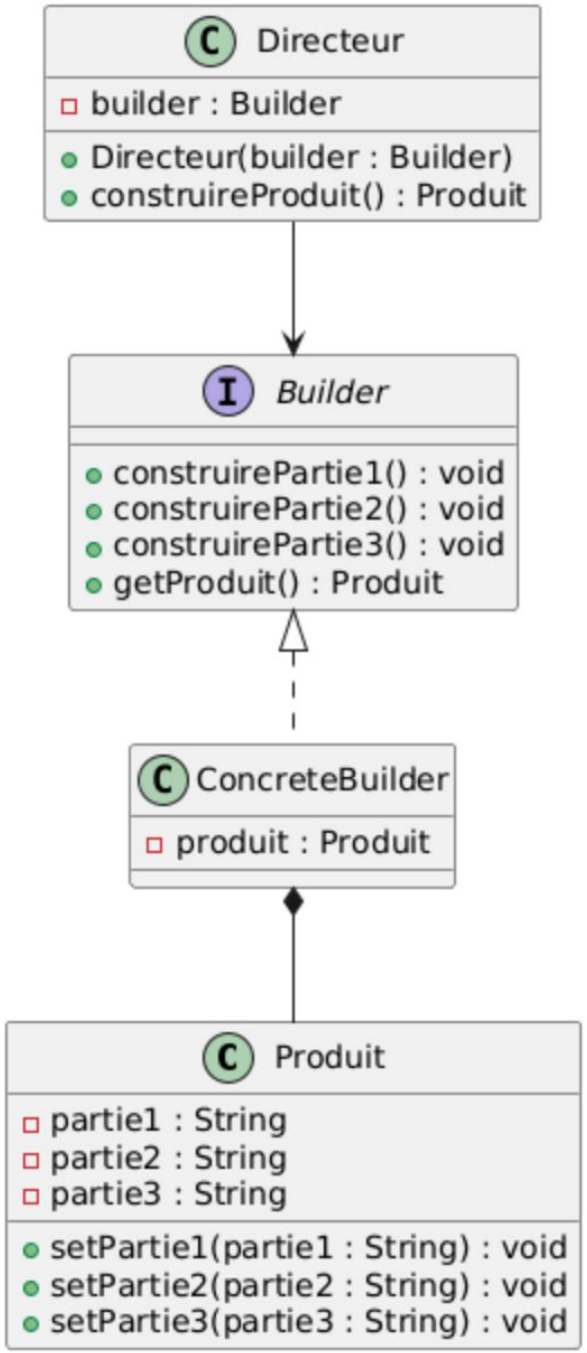
    public Produit getProduit() {
        return produit;
    }
}
```

```java
// Directeur
public class Directeur {
    private Builder builder;

    public Directeur(Builder builder) {
        this.builder = builder;
    }

    public Produit construireProduit() {
        builder.construirePartie1();
        builder.construirePartie2();
        builder.construirePartie3();
        return builder.getProduit();
    }
}
```

```java
// Utilisation
public class Main {
    public static void main(String[] args) {
        Builder builder = new ConcreteBuilder();
        Directeur directeur = new Directeur(builder);
        Produit produit = directeur.construireProduit();
        System.out.println(produit);
    }
}
```



Builder

Avantages :

- Séparation des préoccupations : Sépare la construction d'un objet complexe de sa représentation, rendant le code plus lisible et maintenable.
- Réutilisabilité : Le même code de construction peut être utilisé pour créer différentes représentations d'un objet.
- Contrôle fin : Permet de contrôler étape par étape la création d'un objet, ce qui est utile pour les objets complexes.

Builder

Inconvénients :

- Complexité accrue : ajoute de la complexité en introduisant de nouvelles classes pour le builder et le directeur.

- Verbosité : peut rendre le code plus verbeux en raison des nombreuses étapes de construction.

Prototype

Définition

- Le pattern **Prototype** est un **pattern de création** qui permet de créer de nouveaux objets en copiant des instances existantes (prototypes). Ce pattern est utilisé pour éviter le coût de la création d'objets à partir de zéro.

- Il s'agit de créer des objets basés sur un prototype, plutôt que d'utiliser une instanciation directe.

Prototype

Cas d’utilisation

- Lorsqu'il est coûteux ou complexe de créer une nouvelle instance d'une classe.
- Lorsque le système doit être indépendant du type des objets créés.
- Pour éviter des hiérarchies de classes de constructeurs parallèles ou pour éviter de créer des sous-classes pour chaque type de produit.

Exemple

```
import java.util.HashMap;
import java.util.Map;

// Interface Prototype
public interface Prototype extends Cloneable {
    Prototype clone();
}

// Classe concrète qui implémente Prototype
public class ConcretePrototype implements Prototype {
    private String state;

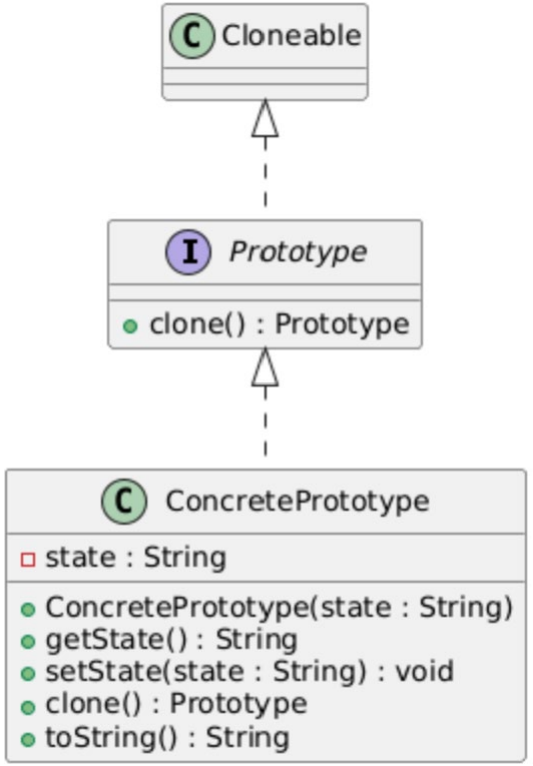
    public ConcretePrototype(String state) {
        this.state = state;
    }

    public String getState() {
        return state;
    }

    public void setState(String state) {
        this.state = state;
    }

    @Override
    public Prototype clone() {
        try {
            return (Prototype) super.clone();
        } catch (CloneNotSupportedException e) {
            e.printStackTrace();
            return null;
        }
    }

    @Override
    public String toString() {
        return "ConcretePrototype [state=" + state + "]";
    }
}
```



Prototype

Avantages

- Efficacité : La création de nouveaux objets en copiant des prototypes peut être plus efficace que l'instanciation directe, surtout pour les objets complexes.

- Réduction des sous-classes : Permet de réduire le nombre de sous-classes nécessaires pour créer des objets similaires.

- Indépendance du type : Le système peut fonctionner avec n'importe quel type d'objet à condition qu'il implémente l'interface Prototype.

Prototype

Inconvénients

- Complexité de la gestion des références : le clonage peut nécessiter une gestion complexe des références d'objet, surtout dans le cas d'objets contenant d'autres objets.

- Support du clonage : toutes les classes ne supportent pas le clonage nativement (implémentation de Cloneable en Java) et peuvent nécessiter une implémentation personnalisée.

Les patterns structurels

- Les **patterns structurels** sont des patterns de conception qui simplifient la composition des objets et des classes pour former des structures plus grandes.

- Ils se concentrent sur la façon dont les classes et les objets sont composés pour former des structures plus complexes tout en gardant ces structures flexibles et efficaces.

Les patterns structurels

Objectifs :

- **Clarté et simplicité** : en utilisant des patterns structurels, les concepteurs peuvent créer des structures de code claires et compréhensibles, facilitant ainsi la maintenance et l'évolution des systèmes.

- **Réutilisation** : ces patterns encouragent la réutilisation de composants existants, ce qui réduit la duplication du code et améliore la cohérence des systèmes.

- **Flexibilité** : ils permettent de créer des systèmes flexibles où les composants peuvent être facilement remplacés ou modifiés sans affecter l'ensemble du système.

Adapter

Définition

- Le pattern **Adapter** est un **pattern structurel** qui permet à des interfaces incompatibles de travailler ensemble.

- Il convertit l'interface d'une classe en une autre interface que le client attend.

Adapter

Cas d'utilisation

- Lorsque vous souhaitez utiliser une classe existante mais que son interface ne correspond pas à celle dont vous avez besoin.

- Lorsque vous souhaitez créer une classe réutilisable qui coopère avec des classes non liées ou des interfaces incompatibles.

- Intégrer une librairie dont l'API ne correspond pas à la vôtre.

Exemple

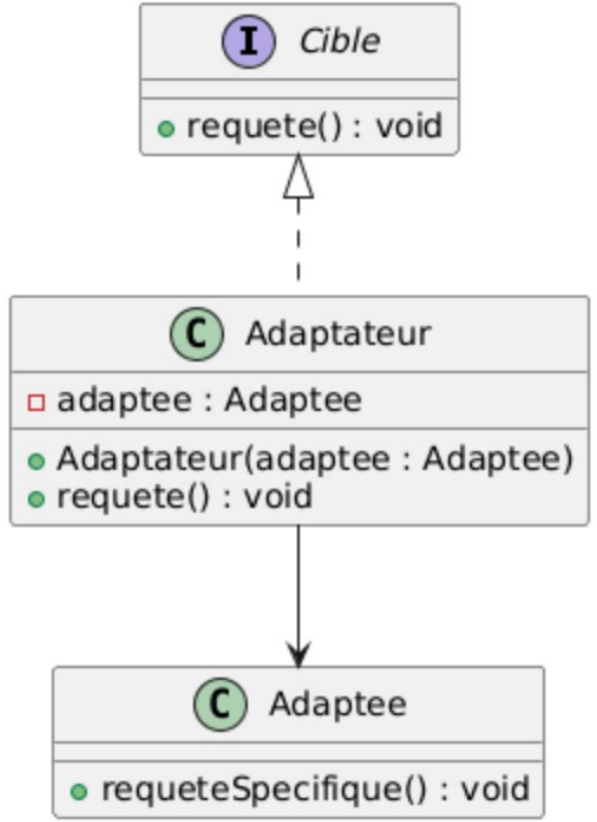
```
// Interface cible que le client utilise
public interface Cible {
    void requete();
}

// Classe existante avec une interface incompatible
public class Adaptee {
    public void requeteSpecifique() {
        System.out.println("Requête spécifique de l'Adaptee");
    }
}
```

```
// Adaptateur qui rend l'Adaptee compatible avec la Cible
public class Adaptateur implements Cible {
    private Adaptee adaptee;

    public Adaptateur(Adaptee adaptee) {
        this.adaptee = adaptee;
    }

    @Override
    public void requete() {
        adaptee.requeteSpecifique();
    }
}
```




```
// Utilisation
public class Main {
    public static void main(String[] args) {
        Adaptee adaptee = new Adaptee();
        Cible cible = new Adaptateur(adaptee);
        cible.requete();  // Affiche "Requête spécifique de l'Adaptee"
    }
}
```

Adapter

Avantages

- **Compatibilité accrue** : Permet d'intégrer des classes existantes avec des interfaces différentes sans modifier leur code.
- **Réutilisabilité** : Facilite la réutilisation de classes existantes dans de nouveaux systèmes.
- **Séparation des préoccupations** : L'adaptateur isole le code spécifique de l'adaptation de l'interface, améliorant ainsi la modularité du code.

Adapter

Inconvénients

- **Complexité supplémentaire** : introduit une couche d'abstraction supplémentaire qui peut augmenter la complexité du code.

- **Peut masquer les incompatibilités** : l'utilisation d'un adaptateur peut parfois masquer des incompatibilités fondamentales entre les systèmes.

Decorator

Définition

- Le pattern **Decorator** est un **pattern structurel** qui permet d'ajouter dynamiquement des responsabilités à un objet.

- Il offre une alternative flexible à la sous-classe pour étendre les fonctionnalités.

Decorator

Cas d’utilisation

- Pour ajouter des responsabilités supplémentaires à des objets individuellement sans affecter d'autres objets de la même classe.
- Pour ajouter des responsabilités à des objets de manière transparente et réversible.
- Lorsque l'héritage est impraticable ou peu flexible.

Exemple

```
// Composant de base
public interface Composant {
    void operation();
}

// Composant concret
public class ComposantConcret implements Composant {
    @Override
    public void operation() {
        System.out.println("ComposantConcret opération");
    }
}
```

```
// Décorateur de base
public abstract class Decorateur implements Composant {
    protected Composant composant;

    public Decorateur(Composant composant) {
        this.composant = composant;
    }

    @Override
    public void operation() {
        composant.operation();
    }
}
```

```
// Décorateur concret A
public class DecorateurConcretA extends Decorateur {
    public DecorateurConcretA(Composant composant) {
        super(composant);
    }

    @Override
    public void operation() {
        super.operation();
        ajoutFonctionnaliteA();
    }

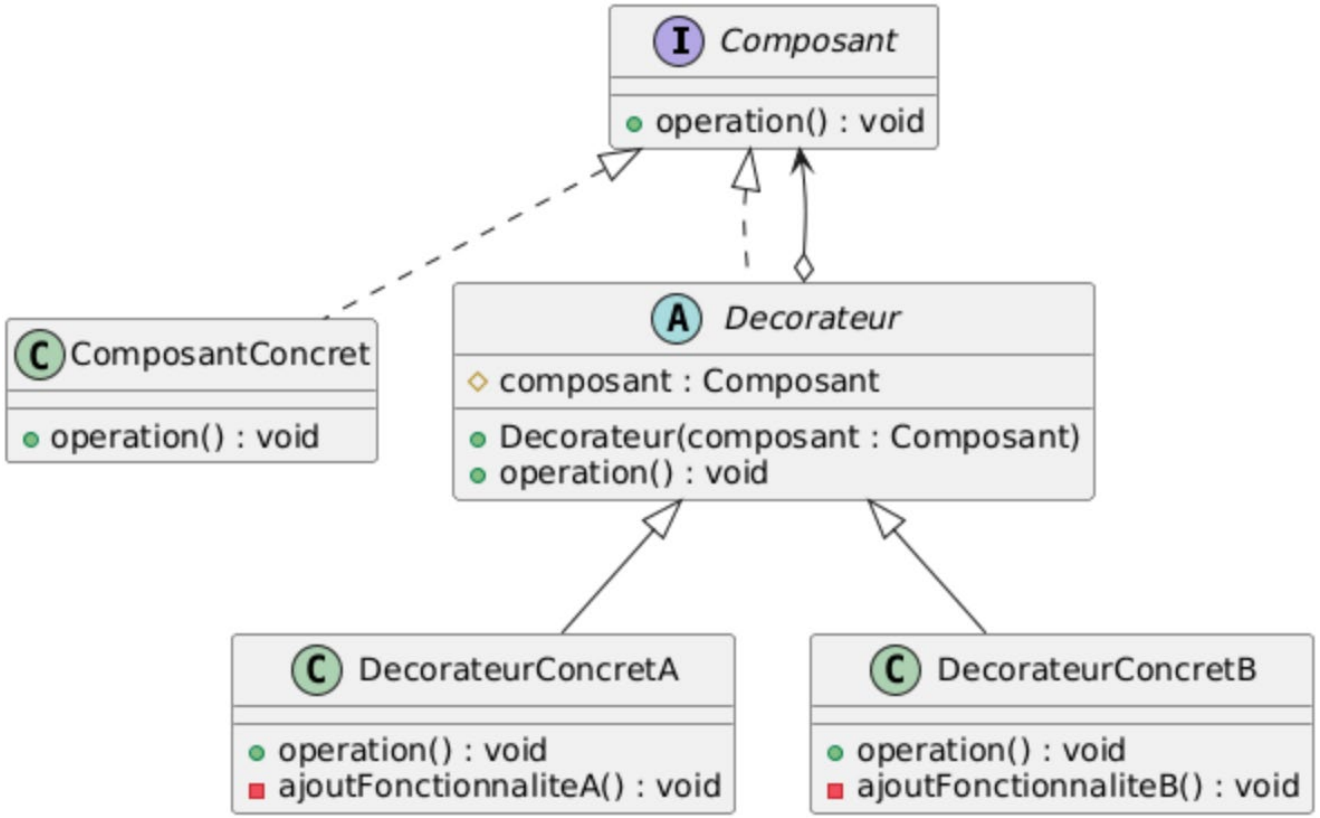
    private void ajoutFonctionnaliteA() {
        System.out.println("Fonctionnalité A ajoutée par DecorateurConcretA");
    }
}
```

```
// Décorateur concret B
public class DecorateurConcretB extends Decorateur {
    public DecorateurConcretB(Composant composant) {
        super(composant);
    }

    @Override
    public void operation() {
        super.operation();
        ajoutFonctionnaliteB();
    }

    private void ajoutFonctionnaliteB() {
        System.out.println("Fonctionnalité B ajoutée par DecorateurConcretB");
    }
}
```

```
// Utilisation
public class Main {
    public static void main(String[] args) {
        Composant composant = new ComposantConcret();
        Composant decorateurA = new DecorateurConcretA(composant);
        Composant decorateurB = new DecorateurConcretB(decorateurA);
        decorateurB.operation();  // Affiche les opérations de ComposantConcret, DecorateurConcretA, et DecorateurConcretB
    }
}
```



Decorator

Avantages

- **Flexibilité** : Permet d'ajouter des fonctionnalités à des objets de manière dynamique et indépendante sans affecter d'autres objets.

- **Réutilisabilité** : Les décorateurs peuvent être combinés de différentes manières pour créer des combinaisons de fonctionnalités.

- **Séparation des préoccupations** : Chaque décorateur a une responsabilité unique, ce qui facilite la maintenance du code.

Decorator

Inconvénients

- **Complexité accrue** : L'utilisation de nombreux décorateurs peut rendre le système plus complexe et difficile à comprendre.

- **Difficile à déboguer** : La chaîne de décorateurs peut rendre le traçage et le débogage du flux d'exécution plus difficiles.

Proxy

Définition

- Le pattern **Proxy** est un **pattern structurel** qui fournit un substitut ou un intermédiaire pour contrôler l'accès à un objet.

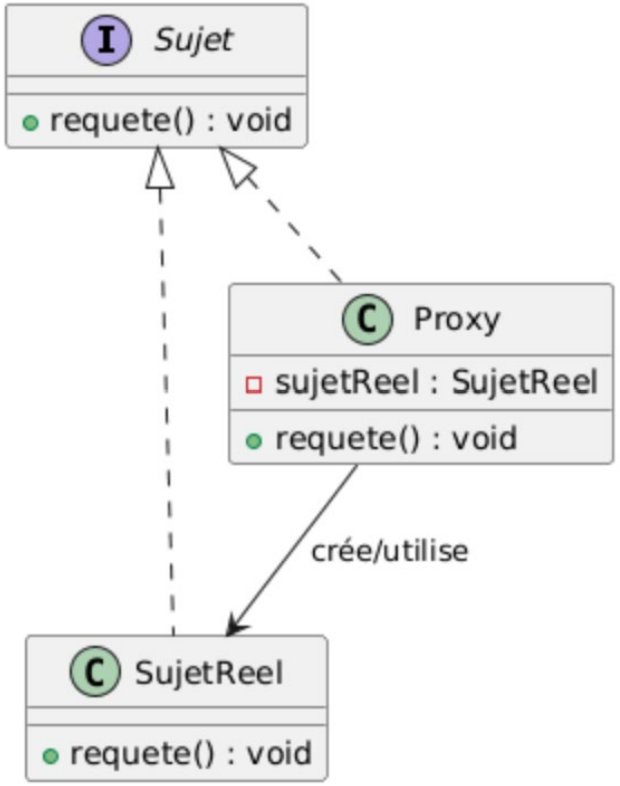
- Il permet de contrôler l'accès à l'objet original en interposant un autre objet (le proxy) qui peut effectuer des actions supplémentaires (comme le contrôle d'accès, le logging, la mise en cache, etc.).

Proxy

Cas d’utilisation

- **Proxy de protection** : Contrôle l'accès à un objet.
- **Proxy virtuel** : Retarde la création et l'initialisation d'objets lourds jusqu'à ce qu'ils soient réellement nécessaires.
- **Proxy de cache** : Fournit un objet local pour représenter un objet distant afin de réduire les appels réseau.
- **Proxy distant** : Représente un objet qui réside dans un autre espace d'adressage (généralement un serveur).

Exemple




```
// Interface sujet
public interface Sujet {
    void requete();
}

// Sujet réel
public class SujetReel implements Sujet {
    @Override
    public void requete() {
        System.out.println("Requête réelle traitée par SujetReel");
    }
}
```

```
// Proxy
public class Proxy implements Sujet {
    private SujetReel sujetReel;

    @Override
    public void requete() {
        if (sujetReel == null) {
            sujetReel = new SujetReel();
        }
        System.out.println("Requête traitée par Proxy");
        sujetReel.requete();
    }
}

// Utilisation
public class Main {
    public static void main(String[] args) {
        Sujet proxy = new Proxy();
        proxy.requete();  // Affiche les messages de Proxy et SujetReel
    }
}
```

Proxy

Avantages

- **Contrôle d'accès** : permet de contrôler et de gérer l'accès à l'objet réel.
- **Performance** : peut améliorer les performances en retardant l'initialisation des objets lourds ou en réduisant les appels réseau.
- **Encapsulation** : cache les détails de l'objet réel et offre une interface uniforme.

Proxy

Inconvénients

- **Complexité accrue** : ajoute une couche supplémentaire qui peut compliquer la conception et la maintenance du système.

- **Coût de l'indirection** : l'utilisation d'un proxy peut introduire une indirection supplémentaire, ce qui peut affecter les performances

Composite

Définition

- Le pattern Composite est un pattern structurel qui permet de composer des objets en structures arborescentes pour représenter des hiérarchies partie-tout.

- Il permet aux clients de traiter de manière uniforme les objets individuels et les compositions d'objets.

Composite

Cas d’utilisation

- Lorsque vous devez représenter des hiérarchies d'objets.
- Lorsque vous voulez que les clients manipulent des objets individuels et des compositions de manière uniforme.
- Pour implémenter des structures de données récursives comme les arbres

Exemple

```
import java.util.ArrayList;
import java.util.List;

// Composant
public interface Composant {
    void operation();
}

// Feuille (élément de base)
public class Feuille implements Composant {
    private String nom;

    public Feuille(String nom) {
        this.nom = nom;
    }

    @Override
    public void operation() {
        System.out.println("Feuille " + nom + " opération");
    }
}
```

```
// Composite (contient d'autres composants)
public class Composite implements Composant {
    private List<Composant> enfants = new ArrayList<>();

    public void ajouter(Composant composant) {
        enfants.add(composant);
    }

    public void retirer(Composant composant) {
        enfants.remove(composant);
    }

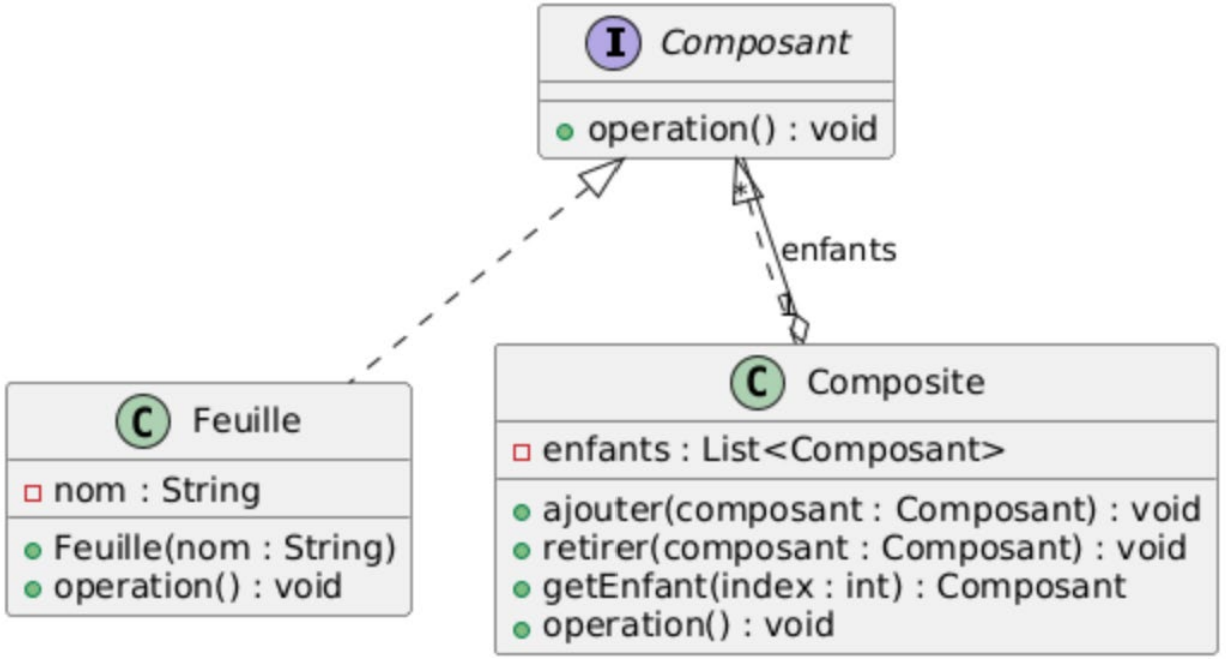
    public Composant getEnfant(int index) {
        return enfants.get(index);
    }

    @Override
    public void operation() {
        System.out.println("Composite opération");
        for (Composant enfant : enfants) {
            enfant.operation();
        }
    }
}
```

```java
// Utilisation
public class Main {
    public static void main(String[] args) {
        Feuille feuille1 = new Feuille("1");
        Feuille feuille2 = new Feuille("2");

        Composite composite = new Composite();
        composite.ajouter(feuille1);
        composite.ajouter(feuille2);

        composite.operation();  // Affiche les opérations de Composite et Feuilles
    }
}
```



Composite

Avantages

- **Simplicité d'utilisation** : Permet aux clients de traiter de manière uniforme les objets individuels et les compositions d'objets.
- **Flexibilité** : Facilite la composition de structures complexes à partir de composants simples.
- **Extensibilité** : Ajouter de nouveaux types de composants est simple et ne nécessite pas de modifications des classes existantes.

Composite

Inconvénients

- **Complexité** : Peut introduire de la complexité si les structures d'objets deviennent très grandes.

- **Gestion de la mémoire** : Peut consommer beaucoup de mémoire en raison des nombreuses références entre les objets.

Facade

Définition

- Le pattern Facade est un pattern structurel qui fournit une interface unifiée et simplifiée à un ensemble d'interfaces dans un sous-système.

- Il permet de réduire la complexité des interactions entre le client et le sous-système en cachant les détails d'implémentation.

Facade

Cas d’utilisation

- Pour simplifier l'utilisation d'une bibliothèque complexe ou d'un ensemble de classes.
- Pour offrir un point d'entrée unique à des sous-systèmes multiples ou à des modules de code.
- Pour découpler le client d'un sous-système complexe.

Exemple

```
// Sous-système complexe
public class SousSystemeA {
    public void operationA() {
        System.out.println("Operation A du SousSystemeA");
    }
}

public class SousSystemeB {
    public void operationB() {
        System.out.println("Operation B du SousSystemeB");
    }
}

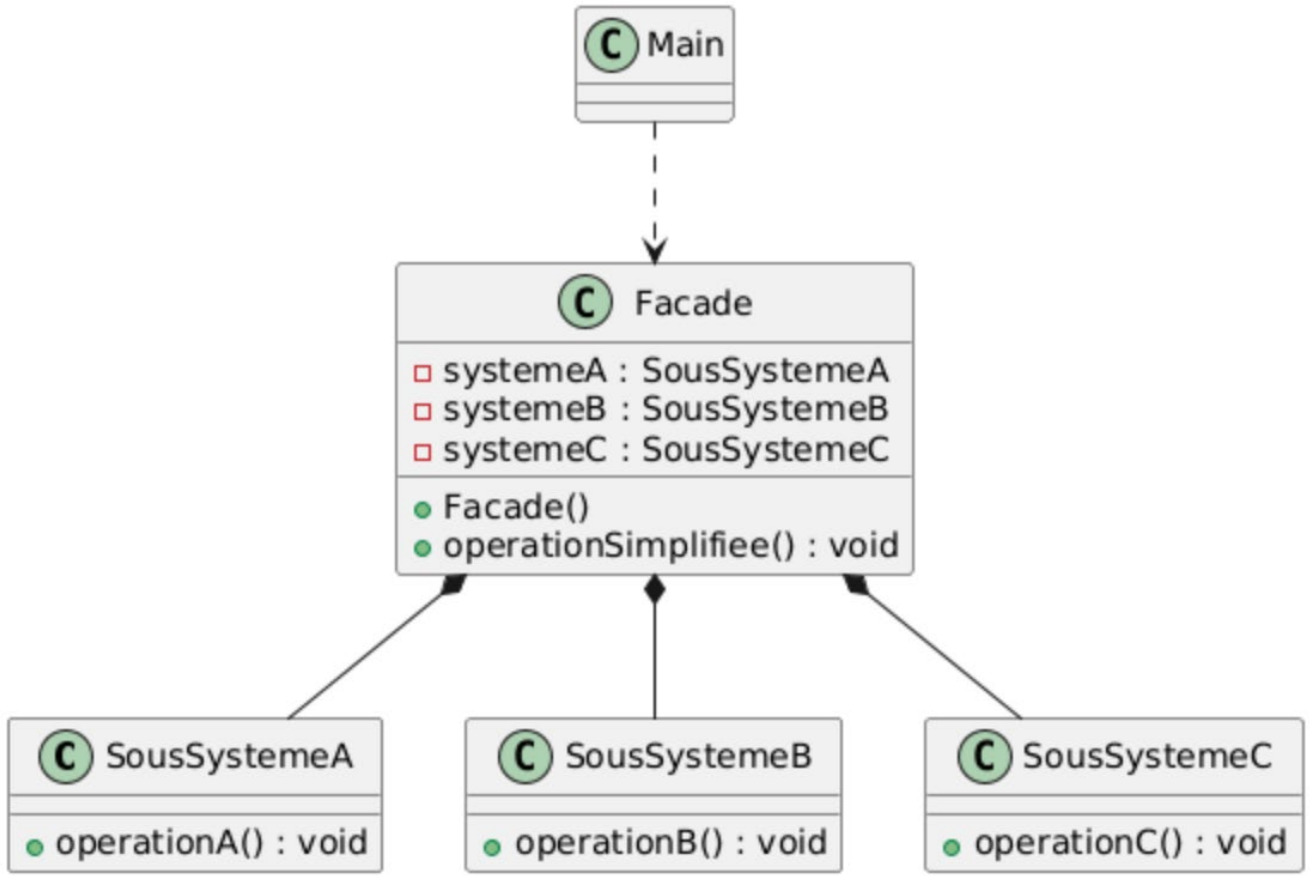
public class SousSystemeC {
    public void operationC() {
        System.out.println("Operation C du SousSystemeC");
    }
}
```

```
// Facade qui simplifie l'accès au sous-système complexe
public class Facade {
    private SousSystemeA systemeA;
    private SousSystemeB systemeB;
    private SousSystemeC systemeC;

    public Facade() {
        systemeA = new SousSystemeA();
        systemeB = new SousSystemeB();
        systemeC = new SousSystemeC();
    }

    public void operationSimplifiee() {
        systemeA.operationA();
        systemeB.operationB();
        systemeC.operationC();
    }
}

// Utilisation
public class Main {
    public static void main(String[] args) {
        Facade facade = new Facade();
        facade.operationSimplifiee();  // Affiche les opérations des sous-systèmes A, B, et C
    }
}
```


Main
Facade
systemeA : SousSystemeA
systemeB : SousSystemeB
systemeC : SousSystemeC
Facade()
operationSimplifiee() : void
SousSystemeA
operationA() : void
SousSystemeB
operationB() : void
SousSystemeC
operationC() : void

Facade

Avantages

- **Réduction de la complexité** : simplifie l'interface avec un sous-système complexe, rendant l'interaction plus intuitive.
- **Découplage** : isolant le client des détails d'implémentation, il permet des changements dans le sous-système sans affecter le client.
- **Facilité d'utilisation** : offre un point d'entrée unique pour plusieurs fonctionnalités du sous-système.

Facade

Inconvénients

- **Masquage des fonctionnalités** : La facade peut ne pas exposer toutes les fonctionnalités du sous-système, limitant ainsi l'accès aux capacités complètes du sous-système.

- **Dépendance accrue** : Le client peut devenir dépendant de la facade, ce qui peut introduire une dépendance forte.

Flyweight

Définition

- Le pattern Flyweight est un pattern structurel qui permet de minimiser l'utilisation de mémoire en partageant autant de données que possible avec des objets similaires.

- Il vise à réduire le coût de création et de gestion d'un grand nombre d'objets en partageant des états communs entre eux.

Flyweight

Cas d’utilisation

- Lorsqu'un grand nombre d'objets similaires sont utilisés et qu'ils consomment une quantité importante de mémoire.
- Lorsqu'il est possible de partager les états communs entre plusieurs objets pour économiser de la mémoire.
- Pour optimiser les performances dans les systèmes où la mémoire est une ressource critique.

Exemple

```
import java.util.HashMap;
import java.util.Map;

// Flyweight Interface
public interface Flyweight {
    void operation(String state);
}

// Concrete Flyweight
public class ConcreteFlyweight implements Flyweight {
    private String intrinsicState;

    public ConcreteFlyweight(String intrinsicState) {
        this.intrinsicState = intrinsicState;
    }

    @Override
    public void operation(String extrinsicState) {
        System.out.println("Intrinsic State = " + intrinsicState + ", Extrinsic State = " + extrinsicState);
    }
}
```

```
// Flyweight Factory
public class FlyweightFactory {
    private Map<String, Flyweight> flyweights = new HashMap<>();

    public Flyweight getFlyweight(String key) {
        if (!flyweights.containsKey(key)) {
            flyweights.put(key, new ConcreteFlyweight(key));
        }
        return flyweights.get(key);
    }

    public int getFlyweightCount() {
        return flyweights.size();
    }
}
```

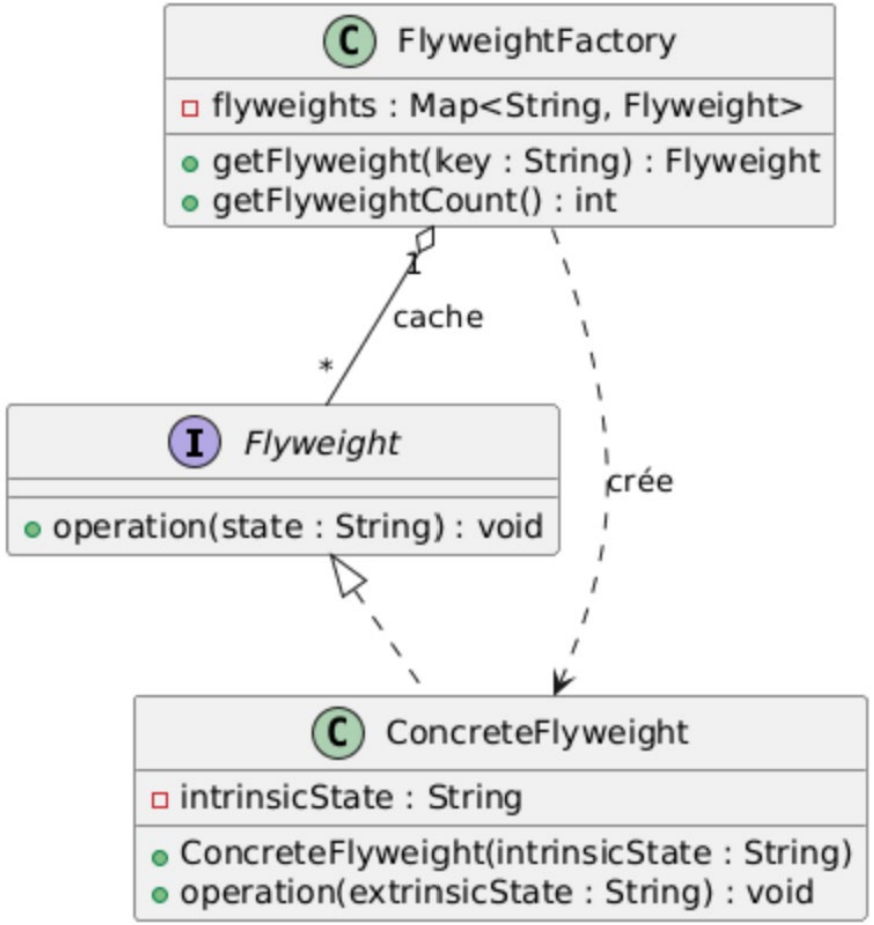
```
// Utilisation
public class Main {
    public static void main(String[] args) {
        FlyweightFactory factory = new FlyweightFactory();

        Flyweight flyweight1 = factory.getFlyweight("A");
        flyweight1.operation("X");

        Flyweight flyweight2 = factory.getFlyweight("B");
        flyweight2.operation("Y");

        Flyweight flyweight3 = factory.getFlyweight("A");
        flyweight3.operation("Z");

        System.out.println("Nombre de flyweights créés: " + factory.getFlyweightCount());
    }
}
```



Flyweight

Avantages

- **Réduction de la consommation de mémoire** : en partageant l'état intrinsèque, le pattern Flyweight peut significativement réduire la consommation de mémoire.

- **Amélioration des performances** : moins d'objets à créer et à gérer peut entraîner des gains de performance.

Flyweight

Inconvénients

- **Complexité accrue** : Introduit une complexité supplémentaire pour gérer le partage des états et la gestion des flyweights.

- **Gestion de l'état extrinsèque** : L'état extrinsèque doit être géré par le client, ce qui peut compliquer la conception.

Bridge

Définition

- Le pattern Bridge est un pattern structurel qui permet de découpler une abstraction de son implémentation afin qu'elles puissent évoluer indépendamment.

- Il sépare l'interface d'une classe de son implémentation, permettant de varier ces deux parties indépendamment.

Bridge

Cas d’utilisation

- Lorsque vous souhaitez éviter un lien étroit entre l'abstraction et son implémentation.
- Lorsque les changements dans l'implémentation de l'abstraction ne doivent pas affecter les clients de l'abstraction.
- Lorsque vous souhaitez combiner plusieurs implémentations possibles de l'abstraction.

Exemple

```
// Abstraction
public abstract class Forme {
    protected Couleur couleur;

    protected Forme(Couleur couleur) {
        this.couleur = couleur;
    }

    public abstract void appliquerCouleur();
}

// Implémentation
public interface Couleur {
    void appliquerCouleur();
}
```

```
// Concrète Implémentation A
public class Rouge implements Couleur {
    @Override
    public void appliquerCouleur() {
        System.out.println("Couleur rouge appliquée.");
    }
}


// Abstraction Raffinée A
public class Cercle extends Forme {
    public Cercle(Couleur couleur) {
        super(couleur);
    }

    @Override
    public void appliquerCouleur() {
        System.out.print("Cercle rempli de couleur ");
        couleur.appliquerCouleur();
    }
}
```

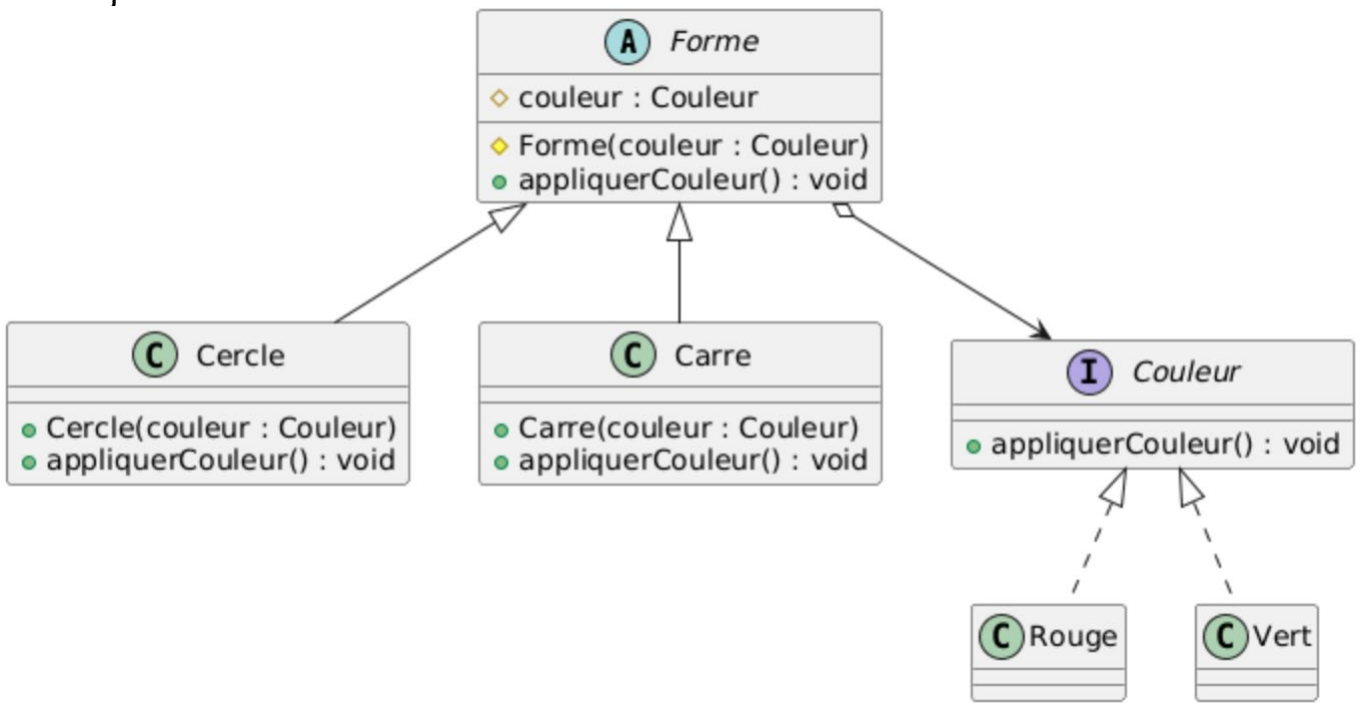
```java
// Concrète Implémentation B
public class Vert implements Couleur {
    @Override
    public void appliquerCouleur() {
        System.out.println("Couleur verte appliquée.");
    }
}
```

```java
// Abstraction Raffinée B
public class Carre extends Forme {
    public Carre(Couleur couleur) {
        super(couleur);
    }

    @Override
    public void appliquerCouleur() {
        System.out.print("Carré rempli de couleur ");
        couleur.appliquerCouleur();
    }
}
```

```java
// Utilisation
public class Main {
    public static void main(String[] args) {
        Forme cercleRouge = new Cercle(new Rouge());
        cercleRouge.appliquerCouleur();

        Forme carreVert = new Carre(new Vert());
        carreVert.appliquerCouleur();
    }
}
```



Bridge

Avantages

- **Indépendance des abstractions et des implémentations** : permet de modifier les abstractions et les implémentations de manière indépendante.
- **Flexibilité accrue** : facilite l'ajout de nouvelles abstractions et implémentations sans modifier les classes existantes.
- **Réduction du couplage** : diminue le couplage entre l'abstraction et son implémentation.

Bridge

Inconvénients

- **Complexité accrue** : introduit plus de classes et d'interfaces, augmentant la complexité globale du code.

- **Peut être excessif pour des structures simples** : peut sembler surdimensionné pour des cas où une simple hiérarchie de classes suffirait.

Les patterns comportementaux

- Les patterns comportementaux sont des patterns de conception qui se concentrent sur les algorithmes et les responsabilités entre les objets.

- Ils décrivent comment les objets interagissent les uns avec les autres, comment ils communiquent et comment les responsabilités sont distribuées entre eux.

Les patterns comportementaux

Objectifs

- **Gérer les interactions et les responsabilités** : faciliter la communication et la distribution des responsabilités entre les objets pour accomplir des tâches complexes.

- **Encapsuler les comportements** : permettre aux objets d'encapsuler des comportements qui peuvent varier indépendamment.

- **Augmenter la flexibilité et la réutilisabilité** : créer des systèmes où les comportements peuvent être facilement modifiés, réutilisés ou étendus.

Strategy

Définition

- Le pattern **Strategy** est un **pattern comportemental** qui définit une famille d'algorithmes, encapsule chacun d'eux et les rend interchangeables.

- Il permet de sélectionner un algorithme à utiliser au moment de l'exécution en définissant une interface commune pour tous les algorithmes.

Strategy

Cas d’utilisation

- Lorsque vous avez plusieurs algorithmes pour accomplir une tâche spécifique et que vous souhaitez pouvoir choisir dynamiquement l'un de ces algorithmes.

- Pour éviter l'utilisation de nombreuses conditions (if-else ou switch-case) pour sélectionner un algorithme.

- Lorsque les algorithmes peuvent être modifiés ou étendus indépendamment des clients qui les utilisent.

Exemple

```
// Interface Strategy
public interface Strategy {
    void execute();
}

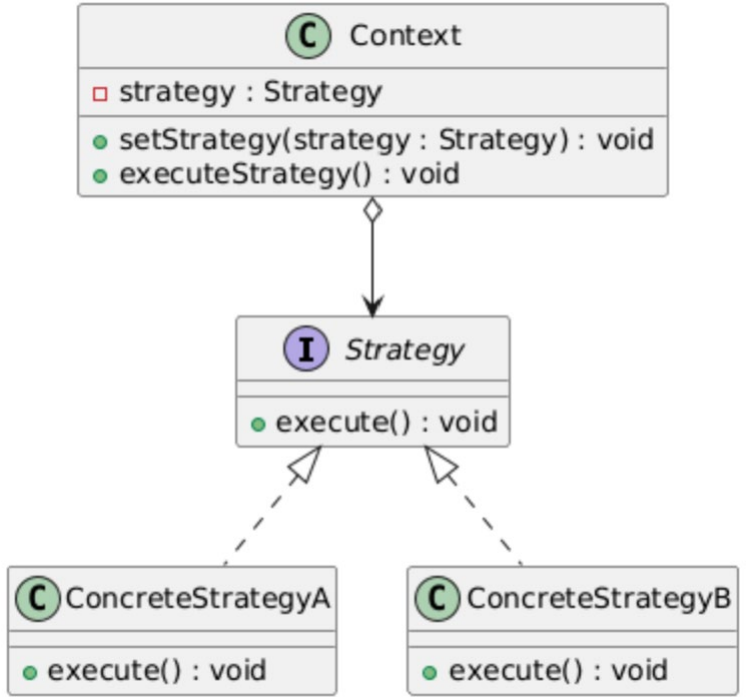
// Implémentations concrètes de Strategy
public class ConcreteStrategyA implements Strategy {
    @Override
    public void execute() {
        System.out.println("Exécution de la stratégie A");
    }
}

public class ConcreteStrategyB implements Strategy {
    @Override
    public void execute() {
        System.out.println("Exécution de la stratégie B");
    }
}
```

```
// Contexte qui utilise une Strategy
public class Context {
    private Strategy strategy;

    public void setStrategy(Strategy strategy) {
        this.strategy = strategy;
    }

    public void executeStrategy() {
        strategy.execute();
    }
}
```



```
// Utilisation
public class Main {
    public static void main(String[] args) {
        Context context = new Context();

        // Utiliser la stratégie A
        context.setStrategy(new ConcreteStrategyA());
        context.executeStrategy();

        // Utiliser la stratégie B
        context.setStrategy(new ConcreteStrategyB());
        context.executeStrategy();
    }
}
```

Strategy

Avantages

- **Élimination des conditions** : Réduit le besoin de nombreuses conditions pour sélectionner un algorithme.
- **Interchangeabilité** : Permet de changer dynamiquement d'algorithme au moment de l'exécution.
- **Extensibilité** : Ajouter de nouvelles stratégies est simple et ne nécessite pas de modification des classes existantes.

Strategy

Inconvénients

- **Complexité accrue** : peut introduire une complexité supplémentaire en ajoutant de nombreuses classes et interfaces.

- **Communication accrue** : le contexte doit connaître les stratégies pour les utiliser, ce qui peut augmenter la complexité de communication.

Observer

Définition

- Le pattern **Observer** est un **pattern comportemental** qui définit une relation de dépendance un-à-plusieurs entre les objets, de sorte que lorsqu'un objet change d'état, tous ses dépendants en sont informés et mis à jour automatiquement.

- Il permet à un objet (le sujet) de notifier plusieurs autres objets (les observateurs) d'un changement d'état sans les coupler de manière étroite.

Observer

Cas d’utilisation

- Lorsqu'un changement dans un objet doit entraîner des changements dans d'autres objets sans les coupler de manière étroite.
- Pour mettre en place une architecture de publication-abonnement où plusieurs objets doivent réagir à des événements produits par un autre objet.
- Lorsqu'un objet doit notifier plusieurs objets sans connaître leur identité exacte.

Exemple

```
import java.util.ArrayList;
import java.util.List;

// Interface Observateur
public interface Observateur {
    void miseAJour(String message);
}

// Classe concrète Observateur
public class ObservateurConcret implements Observateur {
    private String nom;

    public ObservateurConcret(String nom) {
        this.nom = nom;
    }

    @Override
    public void miseAJour(String message) {
        System.out.println(nom + " reçu message: " + message);
    }
}
```

```
// Interface Sujet
public interface Sujet {
    void ajouterObservateur(Observateur observateur);
    void retirerObservateur(Observateur observateur);
    void notifierObservateurs();
}

// Classe concrète Sujet
public class SujetConcret implements Sujet {
    private List<Observateur> observateurs = new ArrayList<>();
    private String etat;

    public void setEtat(String etat) {
        this.etat = etat;
        notifierObservateurs();
    }

    @Override
    public void ajouterObservateur(Observateur observateur) {
        observateurs.add(observateur);
    }

    @Override
    public void retirerObservateur(Observateur observateur) {
        observateurs.remove(observateur);
    }

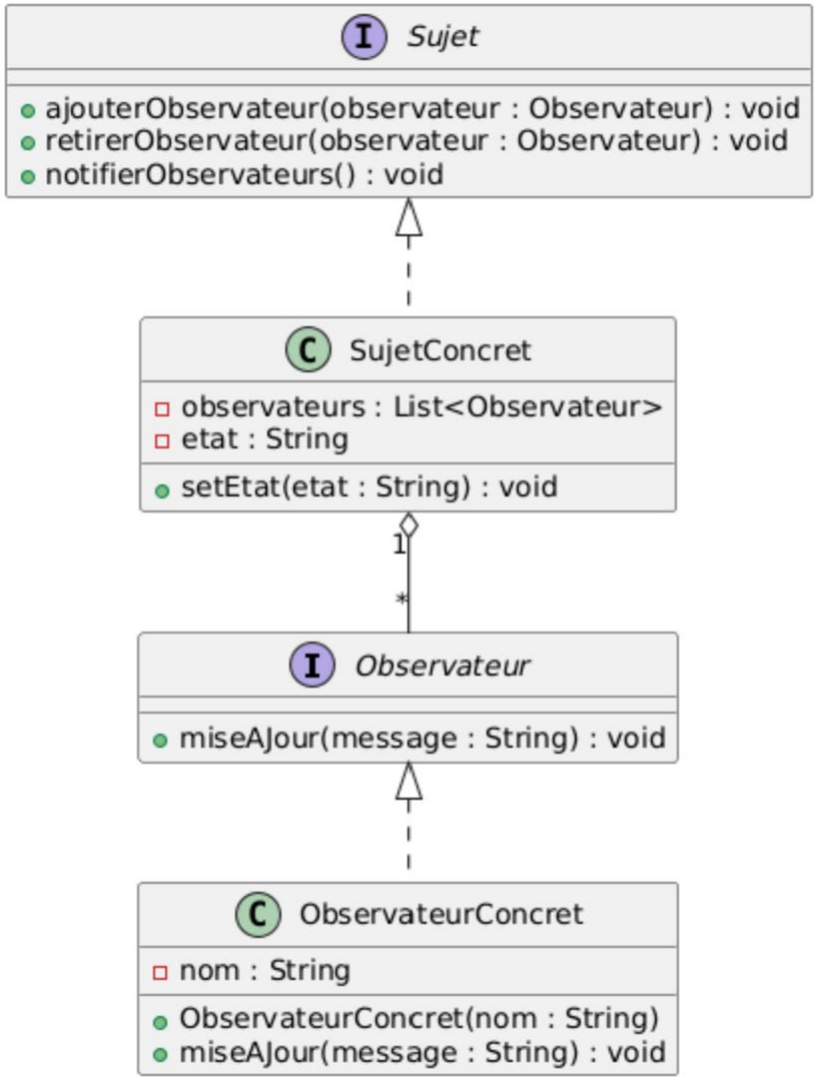
    @Override
    public void notifierObservateurs() {
        for (Observateur observateur : observateurs) {
            observateur.miseAJour(etat);
        }
    }
}
```

```
// Utilisation
public class Main {
    public static void main(String[] args) {
        SujetConcret sujet = new SujetConcret();

        Observateur observateur1 = new ObservateurConcret("Observateur 1");
        Observateur observateur2 = new ObservateurConcret("Observateur 2");

        sujet.ajouterObservateur(observateur1);
        sujet.ajouterObservateur(observateur2);

        sujet.setEtat("Etat 1");
        sujet.setEtat("Etat 2");
    }
}
```



Observer

Avantages

- **Couplage faible** : permet de découpler le sujet des observateurs, favorisant la modularité et la flexibilité.
- **Réutilisabilité** : les observateurs peuvent être réutilisés dans différents contextes sans modification.
- **Évolutivité** : facile d'ajouter de nouveaux observateurs sans modifier le sujet.

Observer

Inconvénients

- **Complexité accrue** : peut introduire de la complexité dans la gestion des observateurs et des notifications.
- **Problèmes de performance** : si le nombre d'observateurs est très élevé, la notification à tous les observateurs peut être coûteuse en termes de performance.
- **Ordre des notifications** : l'ordre dans lequel les observateurs sont notifiés peut être difficile à contrôler.

Command

Définition

- Le pattern **Command** est un **pattern comportemental** qui transforme une demande en un objet autonome contenant toutes les informations nécessaires pour exécuter la demande.

- Il permet de paramétrer les objets avec des opérations à effectuer, de mettre en file d'attente les opérations, et de supporter des opérations annulables.

Command

Cas d’utilisation

- Pour paramétrer les objets avec des opérations à exécuter.
- Pour mettre en file d'attente, enregistrer et annuler des opérations.
- Pour implémenter des opérations qui peuvent être effectuées, annulées ou répétées.
- Lorsque vous voulez découpler l'objet qui invoque l'opération de l'objet qui connaît la façon de la réaliser.

Exemple

```
// Interface Commande
public interface Commande {
    void executer();
}

// Commande concrète pour allumer la lumière
public class AllumerLumiereCommande implements Commande {
    private Lumiere lumiere;

    public AllumerLumiereCommande(Lumiere lumiere) {
        this.lumiere = lumiere;
    }

    @Override
    public void executer() {
        lumiere.allumer();
    }
}
```

```
// Commande concrète pour éteindre la lumière
public class EteindreLumiereCommande implements Commande {
    private Lumiere lumiere;

    public EteindreLumiereCommande(Lumiere lumiere) {
        this.lumiere = lumiere;
    }

    @Override
    public void executer() {
        lumiere.eteindre();
    }
}

// Récepteur
public class Lumiere {
    public void allumer() {
        System.out.println("La lumière est allumée");
    }

    public void eteindre() {
        System.out.println("La lumière est éteinte");
    }
}
```

```
// Invocateur
public class Telecommande {
    private Commande commande;

    public void setCommande(Commande commande) {
        this.commande = commande;
    }

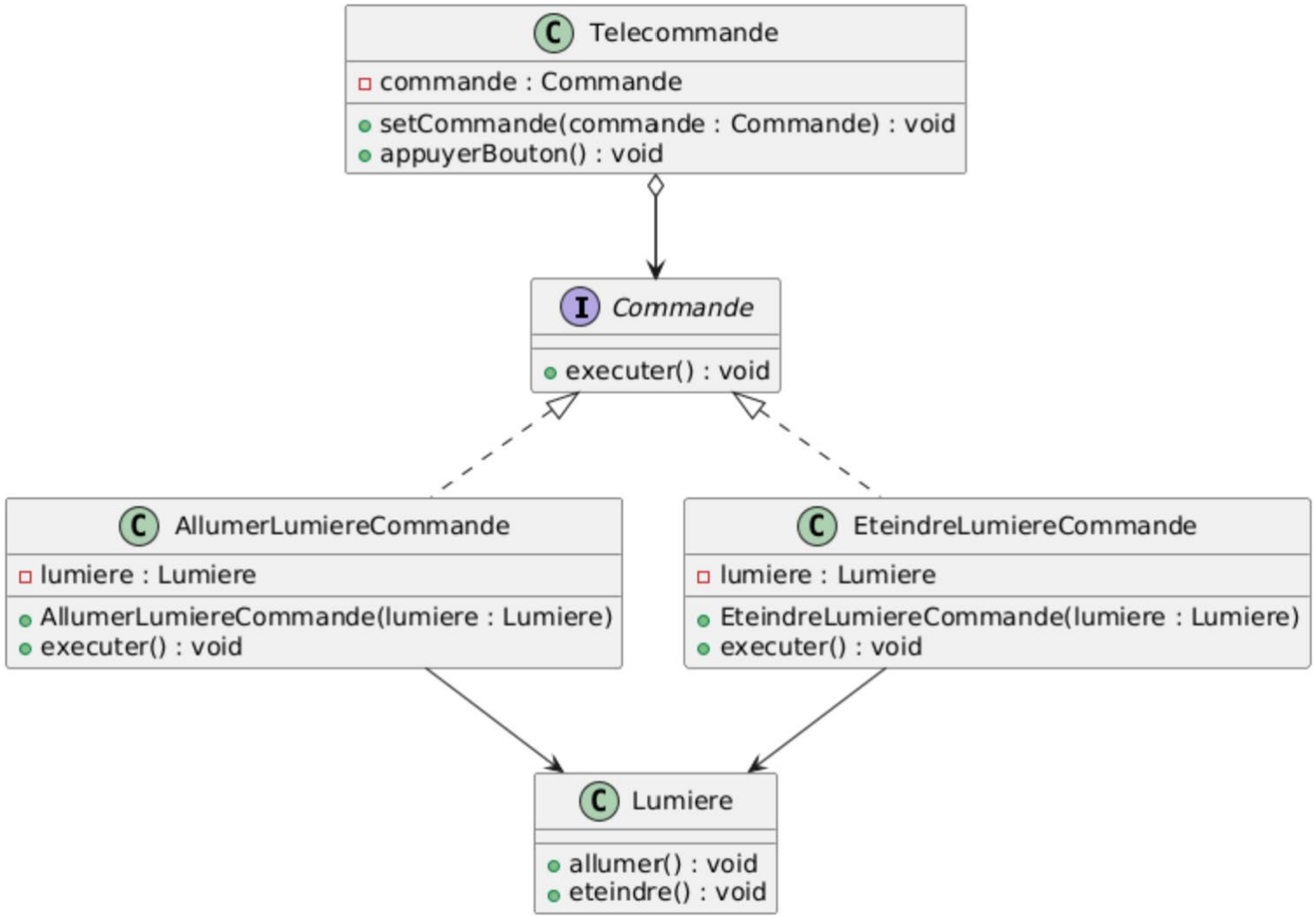
    public void appuyerBouton() {
        commande.executer();
    }
}
```

```
// Utilisation
public class Main {
    public static void main(String[] args) {
        Lumiere lumiere = new Lumiere();
        Commande allumer = new AllumerLumiereCommande(lumiere);
        Commande eteindre = new EteindreLumiereCommande(lumiere);

        Telecommande telecommande = new Telecommande();

        // Allumer la lumière
        telecommande.setCommande(allumer);
        telecommande.appuyerBouton();

        // Éteindre la lumière
        telecommande.setCommande(eteindre);
        telecommande.appuyerBouton();
    }
}
```


Telecommande
commande : Commande
setCommande(commande : Commande) : void
appuyerBouton() : void
Commande
executer() : void
AllumerLumiereCommande
lumiere : Lumiere
AllumerLumiereCommande(lumiere : Lumiere)
executer() : void
EteindreLumiereCommande
lumiere : Lumiere
EteindreLumiereCommande(lumiere : Lumiere)
executer() : void
Lumiere
allumer() : void
eteindre() : void

Command

Avantages

- **Découplage** : découple l'objet qui invoque l'opération de l'objet qui connaît la façon de la réaliser.
- **Historique des commandes** : permet de conserver un historique des opérations effectuées, facilitant ainsi les annulations et les répétitions.
- **Flexibilité** : les commandes peuvent être paramétrées, en file d'attente, enregistrées ou annulées.
- **Facilité d'extension** : il est facile d'ajouter de nouvelles commandes sans modifier les classes existantes.

Command

Inconvénients

- **Complexité accrue** : introduit un grand nombre de classes supplémentaires pour chaque commande.
- **Surcharge pour les opérations simples** : peut sembler excessif pour des opérations très simples.

Iterator

Définition

- Le pattern **Iterator** est un **pattern comportemental** qui fournit un moyen de parcourir séquentiellement une collection d'objets sans exposer sa représentation sous-jacente.

- Il permet de traverser une collection sans que le client ait besoin de connaître la structure interne de cette collection.

Iterator

Cas d’utilisation

- Lorsque vous avez besoin de traverser différentes structures de collection de manière uniforme.
- Pour permettre à une collection de fournir plusieurs façons de traverser ses éléments.
- Pour décourager les collections d'exposer leurs implémentations internes.

Exemple

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

// Interface Agrégat
public interface Agrégat<T> {
    Iterator<T> createIterator();
}

// Agrégat Concret
public class CollectionConcrète<T> implements Agrégat<T> {
    private List<T> items = new ArrayList<>();

    public void ajouter(T item) {
        items.add(item);
    }

    @Override
    public Iterator<T> createIterator() {
        return new IteratorConcret<>(items);
    }
}
```

```
// Iterator Concret
public class IteratorConcret<T> implements Iterator<T> {
    private List<T> items;
    private int position = 0;

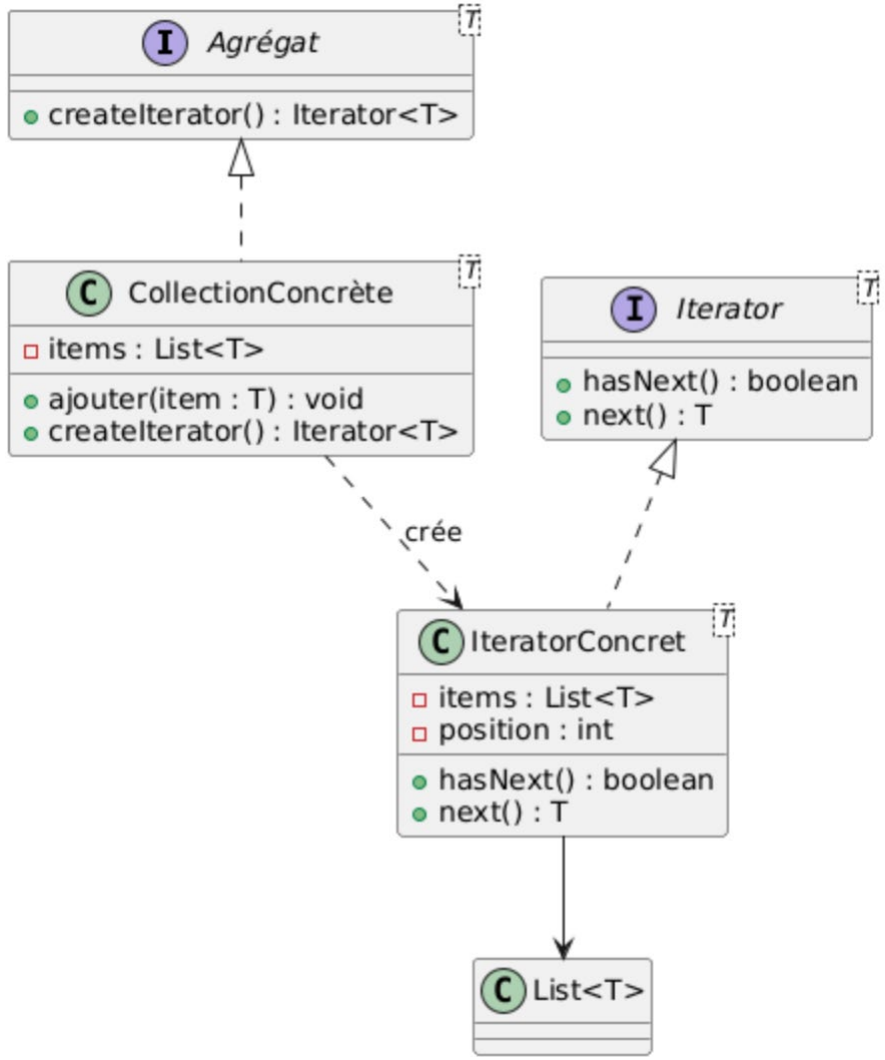
    public IteratorConcret(List<T> items) {
        this.items = items;
    }

    @Override
    public boolean hasNext() {
        return position < items.size();
    }

    @Override
    public T next() {
        return items.get(position++);
    }
}
```

```
// Utilisation
public class Main {
    public static void main(String[] args) {
        CollectionConcrète<String> collection = new CollectionConcrète<>();
        collection.ajouter("Élément 1");
        collection.ajouter("Élément 2");
        collection.ajouter("Élément 3");

        Iterator<String> iterator = collection.createIterator();
        while (iterator.hasNext()) {
            String item = iterator.next();
            System.out.println(item);
        }
    }
}
```



Iterator

Avantages

- **Séparation des responsabilités** : sépare la logique de parcours de la logique de stockage, ce qui améliore la modularité du code.
- **Uniformité** : fournit une interface uniforme pour traverser différentes collections.
- **Encapsulation** : permet de traverser une collection sans exposer sa structure interne.

Iterator

Inconvénients

- **Complexité** : introduit de la complexité supplémentaire avec la création de plusieurs classes pour l'itération.

- **Performances** : peut être moins performant pour certaines opérations de traversée par rapport à des implémentations spécifiques à la collection.

State

Définition

- Le pattern **State** est un **pattern comportemental** qui permet à un objet de changer son comportement lorsque son état interne change.

- Il permet à un objet de paraître changer de classe en modifiant son état interne.

State

Cas d’utilisation

- Lorsqu'un objet doit changer son comportement en fonction de son état.
- Pour éviter d'utiliser de nombreuses conditions if-else ou switch-case pour gérer les transitions d'état.
- Lorsque vous avez besoin de centraliser le code spécifique à un état dans une classe dédiée.

Exemple

```
// Interface Etat
public interface Etat {
    void action(Context context);
}

// Etats Concrets
public class EtatConcretA implements Etat {
    @Override
    public void action(Context context) {
        System.out.println("EtatConcretA : action exécutée.");
        context.setEtat(new EtatConcretB());
    }
}

public class EtatConcretB implements Etat {
    @Override
    public void action(Context context) {
        System.out.println("EtatConcretB : action exécutée.");
        context.setEtat(new EtatConcretA());
    }
}
```

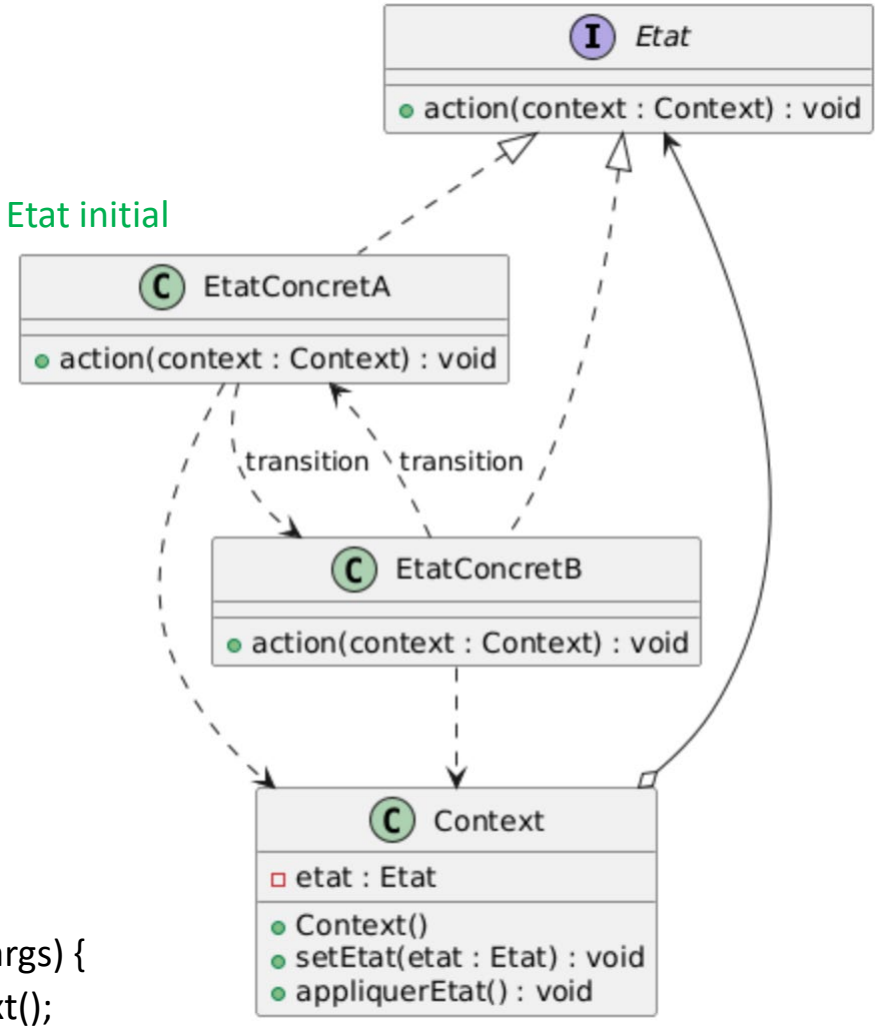
```
// Contexte
public class Context {
    private Etat etat;

    public Context() {
        etat = new EtatConcretA();  // Etat initial
    }

    public void setEtat(Etat etat) {
        this.etat = etat;
    }

    public void appliquerEtat() {
        etat.action(this);
    }
}

// Utilisation
public class Main {
    public static void main(String[] args) {
        Context context = new Context();
        context.appliquerEtat();  // Exécute l'action de l'état A et passe à l'état B
        context.appliquerEtat();  // Exécute l'action de l'état B et passe à l'état A
    }
}
```



State

Avantages

- **Simplification des transitions d'état** : élimine les nombreuses conditions if-else ou switch-case pour gérer les états.
- **Encapsulation** : centralise le code spécifique à un état dans une classe dédiée, améliorant ainsi la lisibilité et la maintenance.
- **Extensibilité** : ajouter de nouveaux états ne nécessite pas de modification des classes existantes.

State

Inconvénients

- **Complexité accrue** : introduit un grand nombre de classes pour chaque état, ce qui peut augmenter la complexité.

- **Communication entre états** : les états peuvent nécessiter une certaine communication entre eux, ce qui peut compliquer la conception.

Template Method

Définition

- Le pattern Template Method est un pattern comportemental qui définit le squelette d'un algorithme dans une méthode, appelant certaines étapes à des sous-classes.

- Il permet aux sous-classes de redéfinir certaines étapes d'un algorithme sans changer la structure de l'algorithme.

Template Method

Cas d’utilisation

- Lorsque vous avez plusieurs classes qui partagent la même structure d'algorithme mais dont certaines étapes doivent être implémentées différemment.
- Pour éviter de dupliquer le code commun dans plusieurs classes.
- Pour permettre à certaines étapes d'un algorithme de varier tout en gardant la structure globale inchangée.

Exemple

```
// Classe abstraite avec la méthode template
public abstract class Jeu {
    // Méthode template
    public final void jouer() {
        demarrer();
        jouerTour();
        arreter();
    }

    // Étapes de l'algorithme à redéfinir par les sous-classes
    protected abstract void demarrer();
    protected abstract void jouerTour();
    protected abstract void arreter();
}
```

```
// Implémentation concrète de Jeu pour le Football
public class Football extends Jeu {
    @Override
    protected void demarrer() {
        System.out.println("Le match de football commence.");
    }

    @Override
    protected void jouerTour() {
        System.out.println("Le match de football est en cours.");
    }

    @Override
    protected void arreter() {
        System.out.println("Le match de football se termine.");
    }
}
```

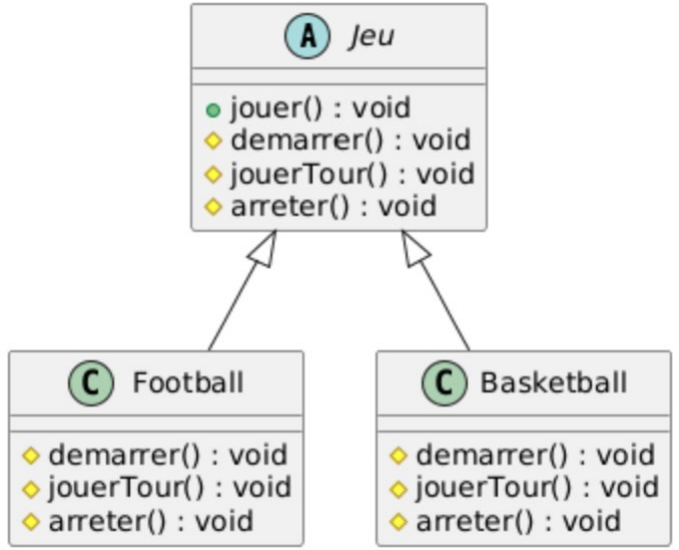
```java
// Implémentation concrète de Jeu pour le Basketball
public class Basketball extends Jeu {
    @Override
    protected void demarrer() {
        System.out.println("Le match de basketball commence.");
    }

    @Override
    protected void jouerTour() {
        System.out.println("Le match de basketball est en cours.");
    }

    @Override
    protected void arreter() {
        System.out.println("Le match de basketball se termine.");
    }
}
```

```java
// Utilisation
public class Main {
    public static void main(String[] args) {
        Jeu football = new Football();
        football.jouer();

        Jeu basketball = new Basketball();
        basketball.jouer();
    }
}
```



Template Method

Avantages

- **Réutilisation du code** : permet de réutiliser le code commun tout en permettant aux sous-classes de personnaliser certaines étapes.
- **Encapsulation des variations** : centralise la structure de l'algorithme tout en encapsulant les variations dans les sous-classes.
- **Facilité de maintenance** : facilite la maintenance et l'extension de l'algorithme en modifiant uniquement les sous-classes.

Template Method

Inconvénients

- **Dépendance aux sous-classes** : les sous-classes sont obligées de redéfinir les étapes spécifiques, ce qui peut conduire à un couplage fort.

- **Risque de changement de comportement** : des modifications dans la classe abstraite peuvent impacter toutes les sous-classes.

Mediator

Définition

- Le pattern Mediator est un pattern comportemental qui définit un objet qui encapsule la manière dont un ensemble d'objets interagit.

- Il permet de réduire les dépendances entre les objets en les forçant à communiquer uniquement via le médiateur.

Mediator

Cas d’utilisation

- Lorsque vous avez un groupe d'objets qui communiquent de manière complexe et que vous souhaitez simplifier ces interactions.

- Pour centraliser la logique de communication entre plusieurs objets afin de réduire le couplage direct entre eux.

- Lorsque vous souhaitez promouvoir la réutilisabilité des objets en les découplant les uns des autres.

Exemple

```
import java.util.ArrayList;
import java.util.List;

// Interface Mediator
public interface Mediator {
    void envoyerMessage(String message, Collegue collegue);
}


// Classe abstraite Collegue
public abstract class Collegue {
    protected Mediator mediator;

    public Collegue(Mediator mediator) {
        this.mediator = mediator;
    }

    public abstract void envoyerMessage(String message);
    public abstract void recevoirMessage(String message);
}
```

```
// Classe concrète Mediator
public class MediatorConcret implements Mediator {
    private List<Collegue> collegues;

    public MediatorConcret() {
        this.collegues = new ArrayList<>();
    }

    public void ajouterCollegue(Collegue collegue) {
        collegues.add(collegue);
    }

    @Override
    public void envoyerMessage(String message, Collegue collegue) {
        for (Collegue c : collegues) {
            if (c != collegue) {
                c.recevoirMessage(message);
            }
        }
    }
}
```

```
// Classe concrète CollegueA
public class CollegueA extends Collegue {
    public CollegueA(Mediator mediator) {
        super(mediator);
    }

    @Override
    public void envoyerMessage(String message) {
        System.out.println("CollegueA envoie message: " + message);
        mediator.envoyerMessage(message, this);
    }

    @Override
    public void recevoirMessage(String message) {
        System.out.println("CollegueA reçoit message: " + message);
    }
}
```

```
// Classe concrète CollegueB
public class CollegueB extends Collegue {
    public CollegueB(Mediator mediator) {
        super(mediator);
    }

    @Override
    public void envoyerMessage(String message) {
        System.out.println("CollegueB envoie message: " + message);
        mediator.envoyerMessage(message, this);
    }

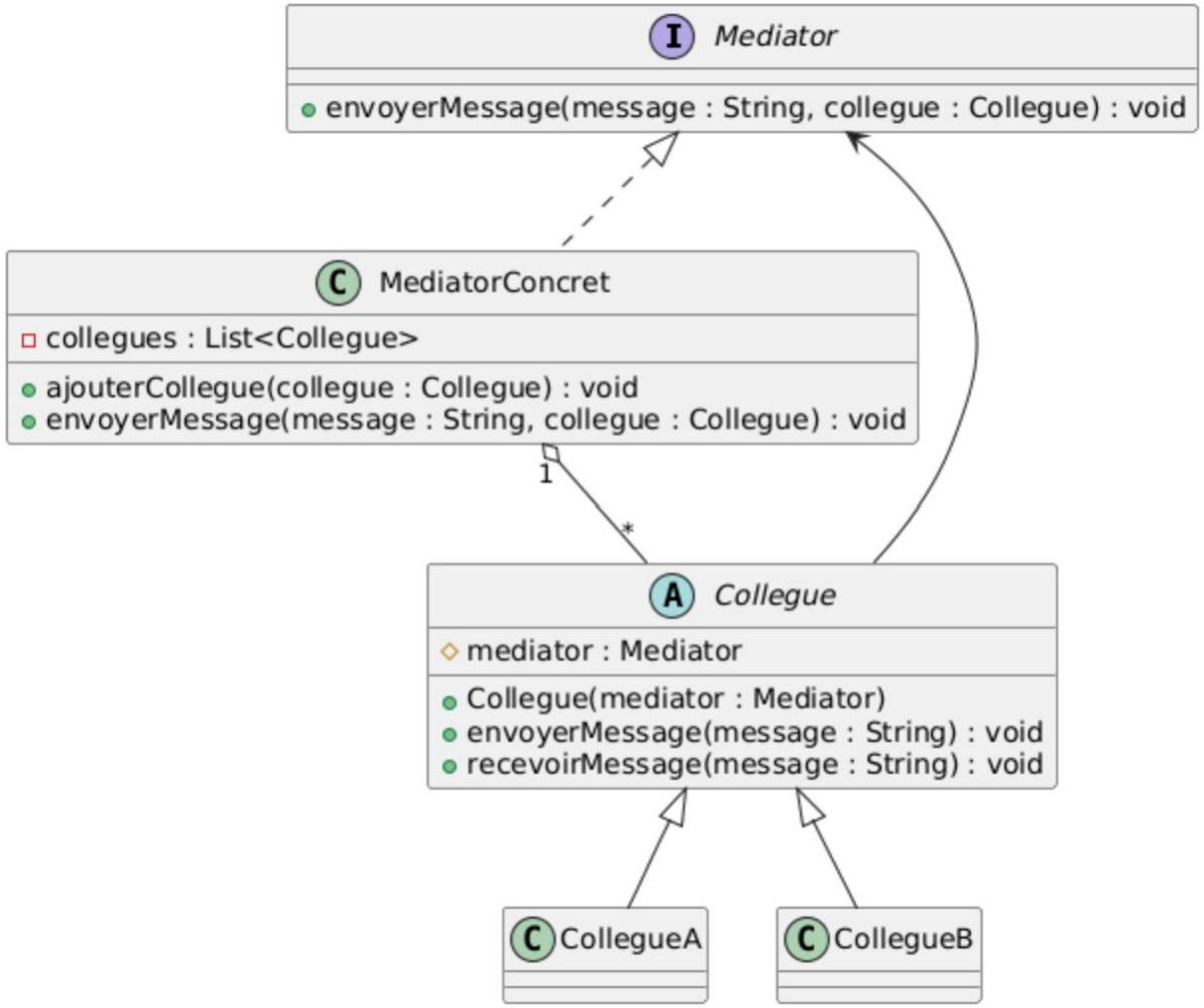
    @Override
    public void recevoirMessage(String message) {
        System.out.println("CollegueB reçoit message: " + message);
    }
}
```

```
// Utilisation
public class Main {
    public static void main(String[] args) {
        MediatorConcret mediator = new MediatorConcret();

        Collegue collegueA = new CollegueA(mediator);
        Collegue collegueB = new CollegueB(mediator);

        mediator.ajouterCollegue(collegueA);
        mediator.ajouterCollegue(collegueB);

        collegueA.envoyerMessage("Bonjour de A");
        collegueB.envoyerMessage("Bonjour de B");
    }
}
```



Mediator

Avantages

- **Réduction du couplage** : élimine les dépendances directes entre les collègues, réduisant ainsi le couplage.
- **Centralisation de la logique** : centralise la logique de communication, ce qui facilite la gestion et la maintenance.
- **Réutilisabilité** : favorise la réutilisabilité des objets en les rendant indépendants les uns des autres.

Mediator

Inconvénients

- **Complexité accrue** : peut introduire une complexité supplémentaire en ajoutant un nouvel objet (le médiateur) à gérer.

- **Dépendance au médiateur** : tous les collègues dépendent du médiateur, ce qui peut créer un point de défaillance unique.

Chain of Responsibility

Définition

- Le pattern **Chain of Responsibility** est un **pattern comportemental** qui permet à plusieurs objets de traiter une demande, chaque objet ayant la possibilité de la traiter ou de la transmettre à un autre objet.

- Il crée une chaîne de récepteurs pour une demande, éliminant ainsi le couplage direct entre l'émetteur et le récepteur.

Chain of Responsibility

Cas d’utilisation

- Lorsque vous avez plusieurs objets pouvant traiter une demande, mais que le destinataire exact n'est pas connu à l'avance.
- Pour éviter le couplage entre l'émetteur d'une demande et ses récepteurs.
- Pour permettre à plusieurs objets de traiter une demande de manière séquentielle jusqu'à ce que l'un d'eux la traite.

Exemple

```
// Classe abstraite Handler
public abstract class Handler {
    protected Handler suivant;

    public void setSuivant(Handler suivant) {
        this.suivant = suivant;
    }

    public abstract void handleRequest(String request);
}

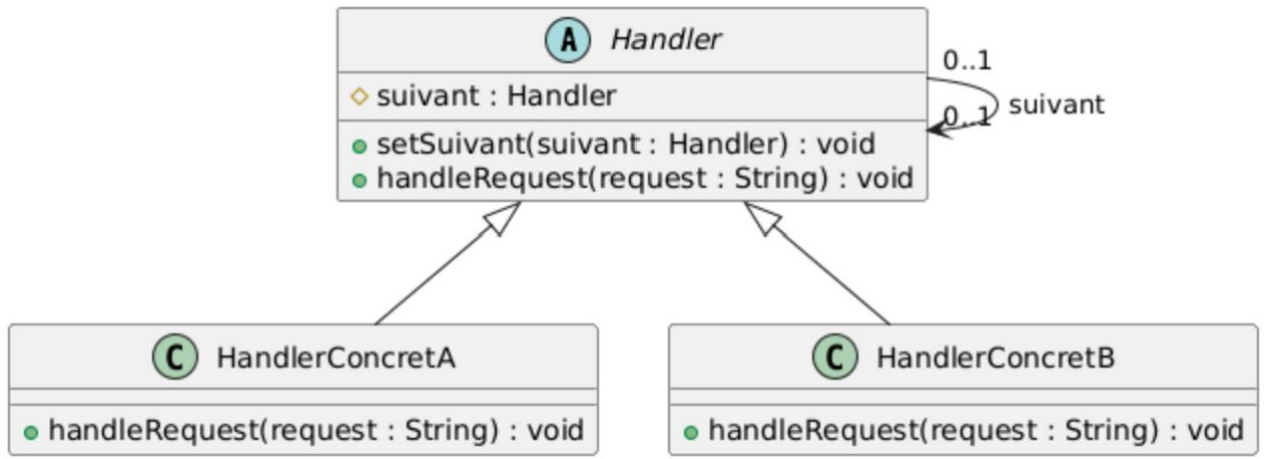
// Handler concret A
public class HandlerConcretA extends Handler {
    @Override
    public void handleRequest(String request) {
        if (request.equals("A")) {
            System.out.println("HandlerConcretA a traité la demande.");
        } else if (suivant != null) {
            suivant.handleRequest(request);
        }
    }
}
```

```
// Handler concret B
public class HandlerConcretB extends Handler {
    @Override
    public void handleRequest(String request) {
        if (request.equals("B")) {
            System.out.println("HandlerConcretB a traité la demande.");
        } else if (suivant != null) {
            suivant.handleRequest(request);
        }
    }
}

// Utilisation
public class Main {
    public static void main(String[] args) {
        Handler handlerA = new HandlerConcretA();
        Handler handlerB = new HandlerConcretB();

        handlerA.setSuivant(handlerB);

        handlerA.handleRequest("A");        // Traité par HandlerConcretA
        handlerA.handleRequest("B");        // Traité par HandlerConcretB
        handlerA.handleRequest("C");        // Non traité
    }
}
```


Handler
suivant : Handler
setSuivant(suivant : Handler) : void
handleRequest(request : String) : void
0..1
suivant
0..1
HandlerConcretA
handleRequest(request : String) : void
HandlerConcretB
handleRequest(request : String) : void

Chain of Responsibility

Avantages

- **Réduction du couplage** : permet de découpler l'émetteur d'une demande de ses récepteurs.
- **Flexibilité** : facilite l'ajout ou la modification de handlers sans affecter les autres composants du système.
- **Responsabilité partagée** : permet de distribuer la responsabilité du traitement de la demande entre plusieurs objets.

Chain of Responsibility

Inconvénients

- **Absence de garantie de traitement** : il n'y a aucune garantie qu'une demande sera traitée si aucun handler approprié n'est trouvé dans la chaîne.

- **Difficulté de débogage** : la chaîne de traitement peut être difficile à suivre et à déboguer en raison de la distribution du traitement entre plusieurs objets.

Memento

Définition

- Le pattern **Memento** est un **pattern comportemental** qui permet de capturer et de sauvegarder l'état interne d'un objet sans violer son encapsulation, afin que l'objet puisse être restauré à cet état plus tard.

- Il permet de revenir à un état précédent de l'objet, souvent utilisé pour implémenter des fonctionnalités de "undo" ou de restauration.

Memento

Cas d’utilisation

- Lorsque vous avez besoin de sauvegarder et de restaurer l'état interne d'un objet.
- Pour implémenter des fonctionnalités de "undo" ou de restauration dans des applications.
- Lorsque vous souhaitez préserver l'encapsulation des données tout en offrant des fonctionnalités de sauvegarde et de restauration.

Exemple

```
// Memento
public class Memento {
    private String state;

    public Memento(String state) {
        this.state = state;
    }

    public String getState() {
        return state;
    }
}
```

```
// Originator
public class Originator {
    private String state;

    public void setState(String state) {
        this.state = state;
        System.out.println("État actuel: " + state);
    }

    public Memento saveStateToMemento() {
        return new Memento(state);
    }

    public void getStateFromMemento(Memento memento) {
        state = memento.getState();
        System.out.println("État restauré: " + state);
    }
}
```

Exemple

```
// Caretaker
public class Caretaker {
    private List<Memento> mementoList = new ArrayList<>();

    public void add(Memento state) {
        mementoList.add(state);
    }

    public Memento get(int index) {
        return mementoList.get(index);
    }
}
```

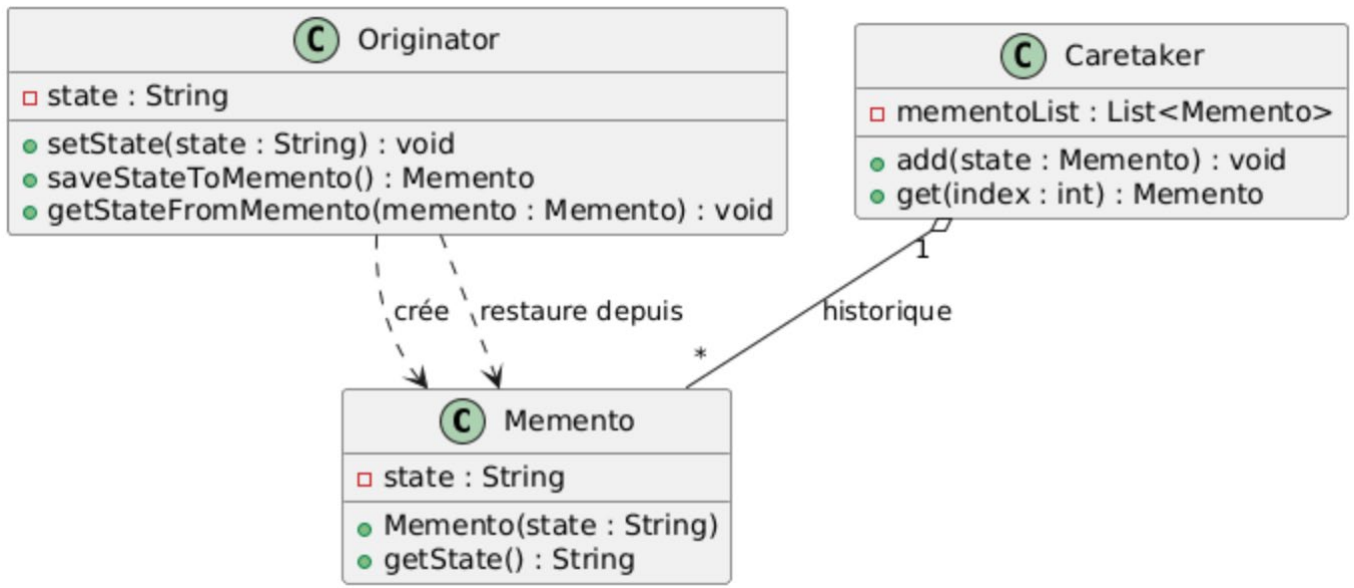
```
// Utilisation
public class Main {
    public static void main(String[] args) {
        Originator originator = new Originator();
        Caretaker caretaker = new Caretaker();

        originator.setState("État 1");
        originator.setState("État 2");
        caretaker.add(originator.saveStateToMemento());

        originator.setState("État 3");
        caretaker.add(originator.saveStateToMemento());

        originator.setState("État 4");
        System.out.println("État actuel: " + originator.getState());

        originator.getStateFromMemento(caretaker.get(0));
        originator.getStateFromMemento(caretaker.get(1));
    }
}
```


Originator
state : String
setState(state : String) : void
saveStateToMemento() : Memento
getStateFromMemento(memento : Memento) : void
Caretaker
mementoList : List<Memento>
add(state : Memento) : void
get(index : int) : Memento
crée
restaure depuis
1
historique
*
Memento
state : String
Memento(state : String)
getState() : String

Memento

Avantages

- **Préservation de l'encapsulation** : le pattern Memento préserve l'encapsulation de l'objet en n'exposant pas ses détails internes aux autres objets.

- **Facilité de restauration** : facilite la sauvegarde et la restauration de l'état d'un objet, utile pour les fonctionnalités de "undo".

- **Historique d'états** : permet de maintenir un historique des états d'un objet, ce qui peut être utile pour l'analyse ou le débogage.

Memento

Inconvénients

- **Coût en mémoire** : la sauvegarde de nombreux états peut consommer une quantité significative de mémoire.
- **Complexité accrue** : introduit des classes supplémentaires pour gérer les mementos, augmentant ainsi la complexité du code.
- **Risque de mauvaise gestion** : si les mementos ne sont pas correctement gérés, ils peuvent conduire à des incohérences ou à des fuites de mémoire.

Intégration de Design Patterns

- **Analyse des besoins** : Identifier les problèmes spécifiques que le design pattern doit résoudre.
- **Conception** : Concevoir l'architecture en intégrant le design pattern choisi. Utiliser des diagrammes UML pour visualiser l'interaction entre les classes et les objets.
- **Implémentation** : Implémenter le design pattern en suivant les meilleures pratiques de codage Java.
- **Test** : Tester le code pour s'assurer que le design pattern résout le problème identifié sans introduire de nouvelles complications.
- **Documentation** : Documenter l'utilisation du design pattern pour faciliter la maintenance future et aider les autres développeurs à comprendre l'architecture.

Bonnes pratiques

- **Utilisation judicieuse** : Ne pas surutiliser les design patterns. Les appliquer uniquement lorsque cela apporte une réelle valeur ajoutée.
- **Simplicité** : Maintenir le code aussi simple que possible. Les design patterns doivent simplifier le développement, pas le compliquer.
- **Refactorisation** : Utiliser les design patterns comme un outil de refactorisation pour améliorer un code existant.
- **Cohérence** : Assurer la cohérence dans l'application des design patterns à travers tout le projet.
- **Collaboratif** : Discuter et partager les choix de design patterns avec l'équipe pour obtenir des feedbacks et assurer une compréhension commune.